

Экономика программной инженерии

Барышникова Марина Юрьевна
МГТУ им. Н.Э. Баумана
Каф. ИУ-7

baryshnikovam@mail.ru

Лекция 6

Экономическая модель разработки ПО. Оценка технико-экономических показателей проекта. Оценка размера программного продукта в строках программного кода. Модель СОСОМО

Несоответствие производительности изначально предполагаемым показателям может иметь две следующие причины: плохая работа или некорректная оценка. В мире программного обеспечения можно получить множество свидетельств плохо выполненной оценки, однако практически невозможно доказать, что персонал не трудился усердно над разрешением проблемы, либо не является в достаточной степени компетентным

Том Де Марко



Технико-экономическое обоснование программного проекта

представляет собой процедуру оценивания трудовых, временных и финансовых ресурсов по созданию программного продукта, соответствующего требованиям заказчика

Объем требуемых ресурсов зависит от:

- ▶ совокупности бизнес-процессов, описывающих предметную область, и их приоритетов для заказчиков
- ▶ требований к функциональной полноте и качеству реализации каждого бизнес-процесса



Две части жизненного цикла программных средств

- ▶ *Первая часть* – включает в себя работы по системному анализу, проектированию, разработке, тестированию и испытаниям базовой версии программного продукта
- ▶ *Вторая часть* - отражает эксплуатацию, сопровождение, модификацию, управление конфигурацией и перенос ПС на иные платформы

Программные продукты являются одними из наиболее сложных объектов, создаваемых человеком, и в процессе их производства творчество специалистов в контексте поиска новых методов, альтернативных решений и способов реализации заданных требований, а также формирование и декомпозиция этих требований составляют значительную часть всех трудозатрат



Исходные данные, используемые для прогнозирования и планирования

- ▶ функции и номенклатура характеристик самого прогнозируемого объекта или процесса
- ▶ характеристики прототипов и пилотных проектов, в некоторой степени подобных планируемому объекту, которые уже завершились и в отношении которых известны необходимые экономические характеристики и можно оценить качество процессов планирования и прогнозирования

Прогнозирование технико-экономических параметров проекта — это анализ и расчёт экономической целесообразности его осуществления, основанный на сравнительной оценке затрат и результатов, эффективности использования, срока окупаемости вложений...

Задача прогнозирования технико-экономических параметров проекта разработки ПО (трудоемкости, стоимости, длительности работ) является достаточно сложной, так как решается в условиях неопределенности. Основная проблема заключается в широком спектре количественных и качественных показателей, которые с различных сторон характеризуют проект разработки ПО, и невысокая достоверность оценки их значений экспертами

При этом наличие корректных прогнозных оценок на этапе принятия решения о целесообразности реализации проекта в условиях ограничений сроков и ресурсов, предлагаемых заказчиком, важно с точки зрения отображения их в условиях будущего договора. Следствием недостатков или отсутствия технико-экономического обоснования является неверная оценка преимуществ новой программной разработки, недооценка роли других конкурирующих предложений, неизбежный перерасход средств и снижение качества программного продукта



Причины для сравнения реальных данных с прогнозируемыми оценками характеристик проекта

- ▶ несовершенство исходных данных при оценивании технико-экономических показателей программных проектов вызывает необходимость периодического пересмотра прогнозных оценок с учетом новой информации, чтобы обеспечить более реальную основу для дальнейшего управления проектом
- ▶ вследствие несовершенства методов оценивания технико-экономических показателей программных проектов следует сравнивать прогнозные оценки этих показателей с действительными значениями, формирующимися в ходе реализации проекта, и использовать эти результаты для улучшения самих методов оценивания
- ▶ программные проекты имеют тенденцию к изменению характеристик и экономических факторов, поэтому для их успеха необходимо своевременно идентифицировать эти изменения и выполнять реалистичное обновление оценок затрат



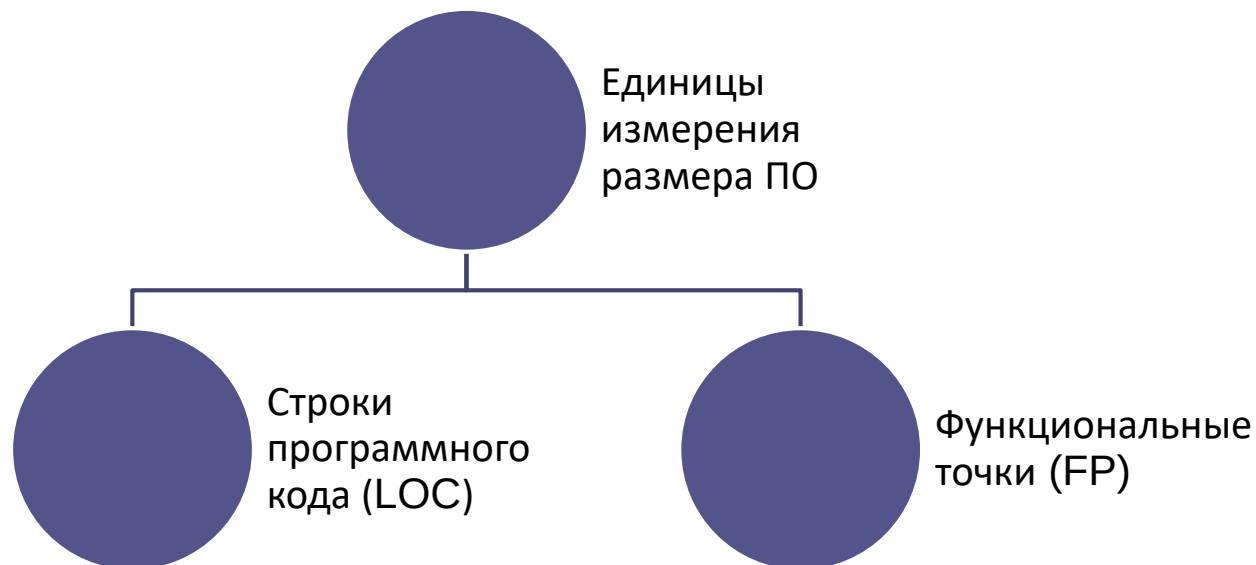
Факторы, повышающие точность оценок

- ▶ цели оценивания технико-экономических показателей должны быть согласованы с потребностями в информации, способствующей принятию решений на соответствующем этапе программного проекта
- ▶ достоверность оценок должна быть сбалансирована для различных компонентов системы и величина уровня неопределенности для каждого компонента должна быть примерно одинаковой, если в процессе принятия решения все компоненты имеют одинаковый вес
- ▶ следует возвращаться к предшествующим целям оценивания технико-экономических показателей и изменять их, когда это необходимо для ответственных решений (прежде всего – касающихся бюджета), принимаемых на ранних этапах и влияющих на следующие этапы



Основные параметры оценки при создании ПО

- ▶ сложность (размеры)
- ▶ трудозатраты на разработку
- ▶ длительность разработки в целом и ее отдельных этапов
- ▶ численность и квалификация специалистов, привлекаемых к созданию ПО



Проблемы использования ЛОС в качестве единицы измерения размера программного продукта

- ▶ число строк исходного кода зависит от уровня мастерства программиста. Фактически, чем выше мастерство программиста, тем меньшим количеством строк кода ему удастся обойтись для реализации определенной функциональной возможности (или функциональности) ПС
- ▶ высокоуровневые языки или языки визуального программирования требуют гораздо меньшего числа строк кода для отражения одной и той же функциональности, чем, например, язык Ассемблера или С. Очевидно, что существует обратная взаимосвязь между уровнем языка и производительностью труда (количеством строк кода в день) программиста
- ▶ фактическое число строк программного кода остается неизвестным до тех пор, пока проект не будет почти завершен. Поэтому ЛОС сложно использовать для предварительной оценки трудозатрат на разработку и построения плана-графика проекта
- ▶ в программистском сообществе не достигнуто соглашение о методе подсчета строк кода. Языковые конструкции, используемые, например, в Visual C++, Ассемблере или SQL существенно различаются. Метод же остается общим для любых приложений, в том числе использующих комбинацию различных языков
- ▶ заказчику сложно понять, каково соотношение указанных им функциональных и нефункциональных (технических) требований к программному продукту и объемов программистской работы



Рекомендации по повышению достоверности ЛОС - оценок

- ▶ убедитесь, что каждая учитываемая строка исходного кода содержит лишь один оператор. Если в одной строке содержатся два выполняемых оператора, разделенных точкой с запятой, то они должны учитываться как две строки. Если же один оператор разбит на несколько «физических» строк, он будет учитываться как одна строка. В языках программирования допускаются различные правила кодирования, но обычно проще определять в строке один оператор, обрабатываемый компилятором или интерпретатором
- ▶ учитывайте все выполняемые операторы. Конечный пользователь может не иметь возможности практически использовать каждый оператор, но все операторы должны поддерживаться данным продуктом, в том числе и утилитами
- ▶ определения данных учитывайте лишь один раз
- ▶ не учитывайте строки, содержащие комментарии
- ▶ не учитывайте отладочный код либо другой временный код (пробное ПО, средства тестирования и пр.)
- ▶ учитывайте каждую инициализацию, вызов или включение макроса (директивы компилятора) в качестве части исходного кода, в которой осуществляется какое-либо действие. Не учитывайте повторно используемые операторы



Пример оценки показателя ЛОС с помощью экспертных оценок

Некоторый объект, отображенный на структуре WBS, по мнению экспертов может занимать от 200 до 400 строк кода, но скорее всего, его размер ближе к 250 строкам

Используя метод PERT, размер объекта можно оценить в 266 строк:

$$(200 + (250 * 4) + 400) / 6 = 266 \text{ LOC}$$

Оценка количества ЛОС по аналогии

Например, у нас имеется уже готовый модуль А, размер которого составляет 2345 LOC. Мы хотим создать новый модуль А', который будет во многом схож с модулем А, но в него будут добавлены некоторые дополнительные свойства. Кроме того, мы знаем как сделать программный код более компактным. В результате этого размер модуля А' может быть оценен в 3000 LOC



Использование языка ассемблера для сравнительного анализа различных проектов

Язык программирования	Basic Assembler
Ассемблер	1
С	2,5
Кобол	3
Фортран	3
Паскаль	3,5
С++	6
Java	6
Ada 95	6,5
Access	8,5
Delphi Pascal	11
CORBA	16

Пусть решается задача по переводу некоторой операционной системы, написанной на языке С и содержащей 50000 строк кода на язык С++

Операционная система, содержащая 50000 строк кода на языке С, эквивалентна 125000 строкам кода на языке ассемблера, которые, будучи переписаны на С++, составят 20833 LOC

Примечание:
 $50000 \cdot 2,5 = 125000$
 $125000 / 6 = 20833$



Преимущества использования ЛОС в качестве единицы измерения

- ▶ Эти единицы широко распространены и могут адаптироваться
- ▶ Они позволяют проводить сопоставление методов измерения размера и производительности в различных группах разработчиков
- ▶ Они непосредственно связаны с конечным продуктом
- ▶ Единицы ЛОС могут быть оценены еще до завершения проекта
- ▶ Оценка размеров ПО производится с учетом точки зрения разработчиков
- ▶ Действия по непрерывному улучшению базируются на количественных оценках. При этом спрогнозированный размер может быть легко сопоставлен с реальным размером на этапе постпроектного анализа. Это позволяет экспертам накапливать опыт и улучшать сами методы оценки
- ▶ Знание размера программного продукта в ЛОС – единицах позволяет применять большинство существующих методов оценки технико-экономических показателей проекта (таких как трудозатраты, длительность проекта, его стоимость и др.)



Недостатки, связанные с применением LOC – оценок

- ▶ Данные единицы измерения сложно применять на ранних стадиях жизненного цикла, когда высок уровень неопределенности
- ▶ Исходные инструкции могут различаться в зависимости от языков программирования, методов проектирования, стиля и способностей программистов
- ▶ Применение методов оценки с помощью количества строк не регламентируется промышленными стандартами, например ISO
- ▶ Разработка ПО может быть связана с большими затратами, которые напрямую не зависят от размеров программного кода (это затраты, связанные с разработкой спецификации требований, подготовкой пользовательской документации и пр., которые не включены в прямые затраты на кодирование)
- ▶ При подсчете LOC – единиц следует различать автоматически сгенерированный код и код, написанный вручную, что сильно затрудняет применение автоматических методов подсчета
- ▶ Генераторы кода зачастую провоцируют его избыточный объем, что может привести к значительным погрешностям в оценке размера ПП
- ▶ Единственным способом получения LOC – оценки является сравнение с аналогичными разработками или экспертные мнения, а эти методы изначально не относятся к числу точных



Способ учета качества программного кода

Программисты могут быть незаслуженно премированы за достижение высоких показателей LOC, если служба менеджмента посчитает это высоким признаком продуктивности

При этом показатели LOC не могут осуществляться для сравнения производительности различных команд разработчиков (либо для нормирования труда) в случае, если использовались разные платформы или типы языков программирования

Количество дефектов / количество строк кода

Софтверные организации склонны вознаграждать программистов, которые: а) пишут много кода; б) исправляют много ошибок. Соответственно, наилучший способ отличиться в таких условиях – это создать большое количество некачественного кода, а потом героически устранять в нем собственные же промахи

Джоэль Спольски (Joel Spolsky) - руководитель компании Fog Creek Software



Экономическая модель разработки ПО

Трудозатраты = (Персонал)(Среда)(Качество)(Размер Процесс)

Размер – размер конечного продукта (для компонентов, написанных вручную), который обычно измеряется числом строк исходного кода или количеством функциональных точек, необходимых для реализации данной функциональности. В это понятие также должны входить и другие создаваемые материалы, такие как документация, совокупность тестовых данных и обучающие материалы

Персонал – возможности персонала, участвующего в разработке ПО, в особенности его профессиональный опыт и знание предметной области проекта. Источниками сложностей могут быть требуемая надежность программного обеспечения, ограничения на производительность и хранение, требуемое повторное использование программных компонентов, а так же опыт работы программистов с данной средой программирования

Среда – состоит из инструментов и методов, используемых для эффективной разработки ПО и автоматизации процесса. Т.е. фактически, это приобретенная или потерянная эффективность вследствие уровня автоматизации процесса (большой уровень автоматизации приводит к уменьшению усилий и повышению эффективности)

Качество – требуемое качество продукта, что включает в себя его функциональные возможности, производительность, надежность и адаптируемость

Процесс – особенности процесса, используемого для получения конечного продукта, в частности, его способность избегать непроизводительных видов деятельности: переделок, бюрократических проволочек, затрат на взаимодействие



Алгоритм оценки затрат на разработку ПО

- ▶ оценка размера разрабатываемого продукта
- ▶ оценка трудозатрат (трудоемкости) в человеко-месяцах или человеко-часах
- ▶ оценка продолжительности проекта в календарных месяцах
- ▶ оценка стоимости проекта

Основным интегральным экономическим показателем каждого программного проекта, отражающим суммарные затраты интеллектуального труда специалистов на производство программного продукта, является трудоемкость (трудозатраты)

Полный анализ и оптимизацию суммарных затрат на проект целесообразно проводить на всем протяжении жизненного цикла, при этом в ряде случаев очень важно учитывать в том числе затраты на сопровождение и эксплуатацию программного продукта

Эти виды затрат характеризуются значительной неопределенностью из-за сложности прогнозирования длительности жизненного цикла, требований качества, степени модификации программ и затрат на сопровождение



Модель оценки стоимости COSCOMO (COnstructive COst MOdel — конструктивная модель стоимости)

Трудозатраты = $C1 * EAF * (\text{Размер})^{P1}$

Время = $C2 * (\text{Трудозатраты})^{P2}$

Трудозатраты (работа) — количество человеко-месяцев;

C1 — масштабирующий коэффициент

EAF — уточняющий фактор, характеризующий предметную область, персонал, среду и инструментарий, используемый для создания рабочих продуктов процесса

Размер — размер конечного продукта (кода, созданного человеком), измеряемый в исходных инструкциях (DSI, delivered source instructions), которые необходимы для реализации требуемой функциональной возможности

P1 — показатель степени, характеризующий экономию при больших масштабах, присущую тому процессу, который используется для создания конечного продукта; в частности, способность процесса избегать непроизводительных видов деятельности (доработок, бюрократических проволочек, накладных расходов на взаимодействие)

Время — общее количество месяцев

C2 — масштабирующий коэффициент для сроков исполнения

P2 — показатель степени, который характеризует инерцию и распараллеливание, присущие управлению разработкой ПО



Допущения модели COSOMO

- ▶ Исходные инструкции конечного продукта включают в себя все (кроме комментариев) строки кода, обрабатываемого компьютером
- ▶ Начало жизненного цикла проекта совпадает с началом разработки продукта, окончание — совпадает с окончанием приемочного тестирования, завершающего стадию интеграции и тестирования
- ▶ Работа и время, затрачиваемые на анализ требований, оцениваются отдельно, как дополнительный процент от разработки в целом
- ▶ Виды деятельности включают в себя только работы, направленные непосредственно на выполнение проекта
- ▶ Человеко-месяц состоит из 152 часов
- ▶ Проект управляется надлежащим образом, в нем используются стабильные требования



Режимы модели COSOMO

Название режима	Размер проекта	Описание	Среда разработки
Обычный	До 50 KLOC	Некрупный проект разрабатывается небольшой командой, для которой нехарактерны нововведения, разработчики знакомы с инструментами и языком программирования	Стабильная
Промежуточный	50 – 500 KLOC	Относительно небольшая команда занимается проектом среднего размера, при этом в процессе разработки необходимы определенные инновации	Среда характеризуется незначительной нестабильностью
Встроенный	Более 500 KLOC	Большая команда разработчиков трудится над крупным проектом, в котором необходим значительный объем инноваций	Среда состоит из множества нестабильных элементов



Формулы для оценки основных работ и сроков

Обычный вариант

Трудозатраты = $3,2 * EAF * (\text{Размер})^{1,05}$

Время (в месяцах) = $2,5 * (\text{Трудозатраты})^{0,38}$

Промежуточный вариант

Трудозатраты = $3,0 * EAF * (\text{Размер})^{1,12}$

Время (в месяцах) = $2,5 * (\text{Трудозатраты})^{0,35}$

Встроенный вариант

Трудозатраты = $2,8 * EAF * (\text{Размер})^{1,2}$

Время (в месяцах) = $2,5 * (\text{Трудозатраты})^{0,32}$

Трудозатраты (работа) — количество человеко-месяцев

EAF — результат учета 15 уточняющих факторов (см. таблицу)

Размер — число исходных инструкций конечного продукта (измеряемое в тысячах строк кода KLOC)

Примечание: Множители и показатели степени приведены для модели COCOMO 81: Intermediate Model, которая известна как промежуточная модель



Значение драйверов затрат в модели COSOMO

Идентификатор	Уточняющий фактор работ	Диапазон изменения параметра	Очень низкий	Низкий	Номинальный	Высокий	Очень высокий
Атрибуты программного продукта							
RELY	Требуемая надежность	0,75-1,40	0,75	0,86	1,0	1,15	1,4
DATA	Размер базы данных	0,94-1,16		0,94	1,0	1,08	1,16
CPLX	Сложность продукта	0,70-1,65	0,7	0,85	1,0	1,15	1,3
Атрибуты компьютера							
TIME	Ограничение времени выполнения	1,00-1,66			1,0	1,11	1,50,
STOR	Ограничение объема основной памяти	1,00-1,56			1,0	1,06	1,21
VIRT	Изменчивость виртуальной машины	0,87-1,30		0,87	1,0	1,15	1,30
TURN	Время реакции компьютера	0,87-1,15		0,87	1,0	1,07	1,15
Атрибуты персонала							
ACAP	Способности аналитика	1,46-0,71	1,46	1,19	1,0	0,86	0,71
AEXP	Знание приложений	1,29-0,82	1,29	1,15,	1,0	0,91	0,82
PCAP	Способности программиста	1,42-0,70	1,42	1,17	1,00	0,86	0,7
VEXP	Знание виртуальной машины	1,21-0,90	1,21	1,1	1,0	0,9	
LEXP	Знание языка программирования	1,14-0,95	1,14	1,07	1,0	0,95	
Атрибуты проекта							
MODP	Использование современных методов	1,24-0,82	1,24	1,1	1,0	0,91	0,82
TOOL	Использование программных инструментов	1,24-0,83	1,24	1,1	1,0	0,91	0,82
SCED	Требуемые сроки разработки	1,23-1,10	1,23	1,08	1,0	1,04	1,1



Несколько слов о сжатии расписания и драйвере SCED

- ▶ Сжатие расписания — это метод управления проектом, используемый для сокращения продолжительности проекта без ущерба для его объема или качества. Потребность в нем возникает в связи с обновленной датой поставки, новой возможностью или задержкой графика.
 - ▶ Одним из способов ускорить время выполнения проекта является выделение на него большего количества ресурсов. Однако следует помнить, что любые дополнительные затраты, связанные с ускорением проекта, сопоставляются с любыми потенциальными выгодами от его более раннего завершения.
 - ▶ Еще один способ сжать расписание – это запараллелить выполнение действий или этапов, которые изначально были запланированы последовательно. Действия могут перекрываться, начинаться раньше, чем предлагалось, а также могут объединяться в расписании. Но этот процесс добавляет риск к графику и должен выполняться осторожно
 - ▶ Наконец еще один способ сжать график – это перераспределить ресурсы с работ, которые не находятся на критическом пути, на те, которые находятся на критическом пути. Но это работает только в том случае, если такая возможность есть
- ▶ Драйвер SCED (ограничение сроков разработки) определяет значимость даты поставки продукта. Высокая степень значимости подразумевает, что продукт желательно или необходимо поставить как можно раньше
 - ▶ Значение драйвера показывает сжатие расписания: очень низкий драйвер соответствует 75% от номинальной длительности; очень высокий – 160% от номинальной длительности

Таким образом, планируя сжатие расписания, следует учитывать, что это вызовет необходимость добавления дополнительных ресурсов в проект, увеличение сверхурочной работы и доплату за более быструю доставку важных компонентов. Было эмпирически доказано, что увеличение степени сжатия увеличивает стоимость проекта в геометрической прогрессии. По этой причине было включено максимальное сжатие в 25%. При превышении этих показателей сжатия проект считается однозначно «провальным»



Распределение работ и времени по стадиям жизненного цикла

<i>Вид деятельности</i>	<i>Трудозатраты (%)</i>	<i>Время (%)</i>
Планирование и определение требований	(+8)	(+36)
Проектирование продукта	18	36
Детальное проектирование	25	18
Кодирование и тестирование отдельных модулей	26	18
Интеграция и тестирование	31	28

Примечание: В основе распределения трудозатрат и времени лежит каскадная модель жизненного цикла



Распределение бюджета по видам работ по созданию ПО

<i>Вид деятельности</i>	<i>Бюджет (%)</i>
Анализ требований	4
Проектирование продукта	12
Программирование	44
Тестирование	6
Верификация и аттестация	14
Канцелярия проекта	7
Управление конфигурацией и обеспечение качества	7
Создание руководств	6
Итого	100



Пример использования модели COSOMO

По контракту с государственной организацией разрабатывается большая, критически важная система (например, для управления электростанцией). Объем программного кода был предварительно оценен в 100 000 строк (100 KSLOC). Используемая технология является новой для разработчиков.

Произвести оценку параметров по методике COSOMO

Все драйверы затрат номинальные, кроме:

Фактор, влияющий на стоимость	Идентификатор	Значение	Значение параметра
Использование программных инструментов	TOOL	Высокое	0,88
Знание приложений	AEXP	Низкое	1,1
Требуемая надежность	RELY	Высокое	1,15
Сложность продукта	CPLX	Высокое	1,15
EAF			1,28



Расчеты по методике COSOMO

- ▶ Трудозатраты = $2,8 * EAF * (\text{Размер})^{1,2} = 2,8 * 1,28 * (100)^{1,2} = 900$ человеко-месяцев на разработку + 72 человеко-месяца на планирование, определение требований = 972 человеко-месяца
- ▶ Время = $2,5 * (\text{Трудозатраты})^{0,32} = 2,5 * (900)^{0,32} = 22$ месяца на разработку + 8 месяцев на планирование, определение требований = 30 месяцев

Учитывая критическую важность и сложность системы, для расчетов использовался встроенный вариант



Распределение работ и времени по этапам жизненного цикла

Вид деятельности	Трудозатраты (чел-мес.)	Время (мес.)
Планирование и определение требований	72	8
Проектирование продукта	162	8
Детальное проектирование	225	4
Кодирование и тестирование отдельных модулей	234	4
Интеграция и тестирование	279	6
ИТОГО:	972	30



Достоинства модели COSOMO

- ▶ Метод является достаточно универсальным и может поддерживать различные режимы и уровни программных разработок
- ▶ При расчетах используются множители и показатели степени, полученные на основе анализа данных большого количества практически реализованных проектов
- ▶ Предложенные драйверы затрат хорошо подгоняются под специфику конкретной организации
- ▶ Точность оценок повышается по мере накопления в организации опыта применения модели
- ▶ Метод снабжен обширной документацией и прост в применении



Недостатки модели COSOMO

- ▶ Все уровни зависят от оценки размера – точность оценки размера оказывает влияние на точность оценки трудозатрат, времени разработки, подбор персонала и оценку производительности
- ▶ Метод основан на каскадной модели жизненного цикла и прежде всего не учитывает изменяемость требований
- ▶ Слишком поверхностное внимание уделено вопросам обеспечения безопасности и надежности
- ▶ Модель не учитывает возможности повторного использования кода, итерационные возвраты по этапам жизненного цикла, объектно-ориентированные технологии разработки ПО





Спасибо за внимание!



Экономика программной инженерии

Барышникова Марина Юрьевна
МГТУ им. Н.Э. Баумана
Каф. ИУ-7

baryshnikovam@mail.ru

Лекция 7

Влияние эффектов повторного использования кода на размер ПО.
Модель COSOMO II

<https://csse.usc.edu/> - Центр системной инженерии
и программного обеспечения имени Боэма



Концепция повторного использования

- ▶ Многие программы происходят от предыдущих версий этих же программ
- ▶ В результате может быть достигнута экономия средств и/или времени, а также повышен уровень качества
- ▶ Повторному использованию подлежат: программный код, тестовый код, тестовые процедуры, документация, дизайн, спецификации требований и др.
- ▶ Программное обеспечение, повторно используемое в полном объеме, помимо собственно программного кода имеет идентичную документацию, идентичные тестовые процедуры и сам тестовый код и только одну копию, поддерживаемую системой управления конфигурацией



Повторное использование кода

- ▶ Новый код – это код, разработанный для нового приложения, который не включает большие порции ранее написанного кода
- ▶ Модифицируемый код – это код, разработанный для предыдущих приложений, который будет пригоден для использования в новом приложении после внесения умеренного объема изменений
- ▶ Повторно используемый код – это код, разработанный для предыдущих приложений, который будет пригодным для новых приложений без внесения каких-либо изменений

Первый этап при оценке систем, в которых возможно повторное использование кода, заключается в отделении нового кода от модифицируемого и повторно используемого кода. Это необходимо, так как интеграция модифицируемого и повторно используемого кода требует дополнительного объема трудозатрат



Рекомендации по выделению повторно используемого кода

- ▶ В качестве оцениваемой единицы выбирается программный модуль (примерный размер около 100 LOC)
- ▶ Если единица не была изменена она принимается за повторно используемый код
- ▶ Если единица была изменена (неважно идет ли речь об исполняемом коде, либо о комментариях) она считается модифицируемой
- ▶ Если объем изменений затрагивает более 50% кода единицы она принимается за новую
- ▶ В модифицируемом коде различают изменения, направленные на устранение дефектов и изменения, направленные на расширение функциональности



Пример, показывающий распределение новых модифицируемых и повторно используемых строк кода

Элемент	LOC для нового кода	LOC для модифицируемого кода	LOC для повторно используемого кода	Итого LOC
Компонент 1	1233	0	0	1233
Компонент 2	0	988	0	988
Компонент 3	0	0	781	781
Компонент 4	560	245	0	805
Компонент 5	345	549	420	1314
ИТОГО	2138	1782	1201	5121



Типичные множители повторного использования кода

Степень простоты использования	Повторно используемый код	Модифицируемый код
Простой	10%	25%
Средний	30%	60%
Сложный	40%	75%

Применение множителей повторного использования кода

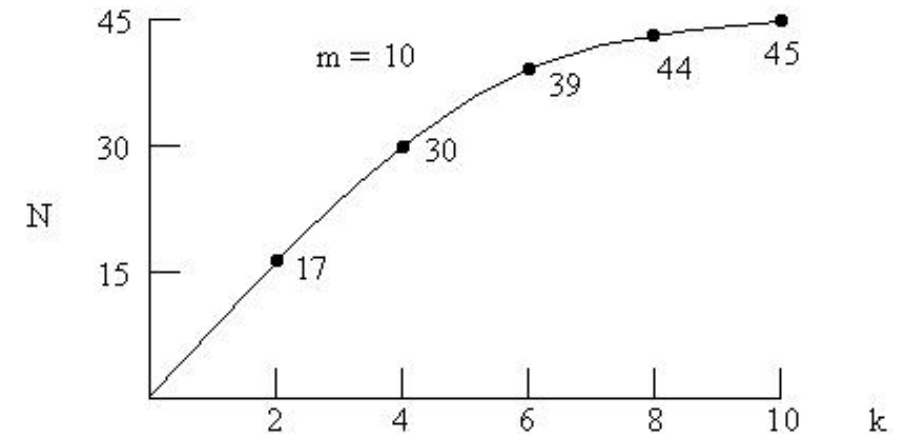
Элемент	LOC для нового кода	LOC для модифицируемого кода	LOC для повторно используемого кода	Итого LOC
Итого	2138	1782	1201	5121
Множитель	100%	60%	30%	
Результат	2138	1069	360	3567



Нелинейные эффекты повторного использования

Проведенный в 80-90-х годах анализ затрат на повторное использование почти 3000 программных модулей показывает, что функция приведенных затрат нелинейная в двух аспектах:

- Как правило, затраты на оценку, выбор и интеграцию компонента многократного использования составляют около 5%.
- Небольшие модификации приводят к непропорционально большим затратам. В первую очередь это связано с двумя факторами: стоимостью понимания программного обеспечения, подлежащего модификации, и относительной стоимостью проверки интерфейса



если модифицировать k из m программных модулей, количество N необходимых проверок интерфейса модуля равно $N = k * (m-k) + k * (k-1)/2$

Как только происходит переход от немодифицированного повторного использования («черный ящик») к модифицированному программному обеспечению («белый ящик»), человек сталкивается с недостатком понимания программного обеспечения. Проведенные исследования показывают, что 47% усилий по сопровождению программного обеспечения связано с пониманием программного кода, подлежащего модификации

COCOMO II

Проект COCOMO II стартовал в 1995 году в центре по разработке ПО USC при финансовой и технической поддержке большого количества промышленных предприятий, таких как AT&T, BELL Labs, Hewlett-Packard, Rational, Texas Instruments, Lockheed Martin, Motorola, Xerox и др.

Проект имел триединую задачу:

- ▶ разработать модель для оценки стоимости и сроков создания ПО для того жизненного цикла, который будет применяться в конце 20 – начале 21 века
- ▶ разработать базу данных стоимости программных проектов и осуществить инструментальную поддержку методов усовершенствования модели стоимости
- ▶ создать количественную аналитическую схему для оценки технологий создания ПО и их экономического эффекта



Три различные модели оценки стоимости в COSOMO II

- ▶ Модель композиции приложения – это модель, которая подходит для проектов, созданных с помощью современных инструментальных средств. Единицей измерения служит объектная точка
- ▶ Модель ранней разработки архитектуры. Эта модель применяется для получения приблизительных оценок проектных затрат периода выполнения проекта перед тем как будет определена архитектура в целом. В этом случае используется небольшой набор новых драйверов затрат и новых уравнений оценки. В качестве единиц измерения используются функциональные точки либо KSLOC
- ▶ Постархитектурная модель – наиболее детализированная модель COSOMO II, которая используется после разработки архитектуры проекта. В состав этой модели включены новые драйверы затрат, новые правила подсчета строк кода, а также новые уравнения



Характеристики моделей оценки стоимости COCOMO II

Модель композиции приложения	Модель ранней разработки архитектуры	Постархитектурная модель
Грубые входные данные	Ясно понимаемые особенности проекта	Детальное описание проекта
Оценки низкой точности	Оценки умеренной точности	Высокоточные оценки
Приблизительные требования	Ясно понимаемые требования	Стабилизировавшиеся основные требования
Концепция архитектуры	Ясно понимаемая архитектура	Стабильная базовая архитектура



Модель композиции приложения

Данная модель используется на ранней стадии конструирования ПО, когда:

- ▶ рассматривается макетирование пользовательских интерфейсов
- ▶ оценивается производительность
- ▶ определяется степень зрелости технологии

Модель ориентирована на применение объектных точек. Объектная точка — средство косвенного измерения ПО. Подсчет количества объектных точек производится с учетом количества экранов (как элементов пользовательского интерфейса), отчетов и компонентов, требуемых для построения приложения

Модель композиции приложения следует рассматривать как некую рабочую гипотезу, основанную на концепции продукта



Правила подсчета объектных точек

- ▶ количество изображений на дисплее (экранных форм). Простые изображения принимаются за 1 объектную точку, изображения умеренной сложности принимаются за 2 точки, очень сложные изображения принято считать за 3 точки
- ▶ количество представленных отчетов. Для простых отчетов назначаются 2 объектные точки, умеренно сложным отчетам назначаются 5 точек. Написание сложных отчетов оценивается в 8 точек
- ▶ количество модулей, которые написаны на языках третьего поколения (C, Java, Python и пр.) и разработаны в дополнение к коду, написанному на языке программирования четвертого поколения. Каждый модуль на языке третьего поколения считается за 10 объектных точек



Критерии сложности экранов

Сложность экрана зависит от количества:

- ▶ представлений на экране
- ▶ таблиц данных, с которыми экран работает
- ▶ серверов, к которым идёт обращение
- ▶ клиентов (пользователей), использующих экран

Параметр	Простой	Средний	Сложный
Количество представлений (views)	0–1	2–3	≥ 4
Количество таблиц данных	< 3	3–7	> 8
Количество серверов	1	2	≥ 3
Количество клиентов	1–2	3–4	≥ 5



Критерии сложности отчетов

Сложность отчета зависит от количества:

- ▶ секций в отчёте
- ▶ таблиц данных, используемых для генерации
- ▶ серверов, откуда берутся данные
- ▶ клиентов, получающих отчёт

Параметр	Простой	Средний	Сложный
Количество секций	0–1	2–3	≥ 4
Количество таблиц данных	< 3	3–7	> 8
Количество серверов	1	2	≥ 3
Количество клиентов	1–2	3–4	≥ 5



Модель композиции приложения

$NOP = (\text{Объектные точки}) \times [(100 - \%RUSE) / 100]$ – новые объектные точки

$\text{ТРУДОЗАТРАТЫ} = NOP / \text{PROD}$ [чел.-мес.]

PROD - оценка скорости разработки

Модель композиции приложения используется на этапе создания прототипов и анализа осуществимости

Опытность/ возможности разработчика	Зрелость/ возможности среды разработки	PROD
Очень низкая	Очень низкая	4
Низкая	Низкая	7
Номинальная	Номинальная	13
Высокая	Высокая	25
Очень высокая	Очень высокая	50



Пример использования модели композиции приложения

Найти трудозатраты и календарное время работы над следующим проектом: приложение формирует 2 формы средней сложности (запросы к базе данных), 3 отчета высокой сложности и имеет 2 программных модуля на языке 3 уровня. Процент повторного использования кода программы – 10%. Над проектом работает программист средней квалификации, однако имеющий опыт работы в данной предметной области

3. Определим длительность выполнения проекта на уровне композиции приложения:

$$\text{Время} = 3,67 * (\text{Трудозатраты})^{(0,28 + 0,2 * (p-1,01))} = 3,67 * 3,32^{(0,28 + 0,2 * (p-1,01))} = 3,67 * 1,4 = \mathbf{5,12 \text{ (месяца)}},$$

где $p = 1$

1. Найдем количество объектных точек и их суммарную балльную оценку

№	Наименование объекта	Уровень сложности (коэф-нт)	Количество	Число точек
1	Форма	Средний (2)	2	4
2	Отчет	Высокий (8)	3	24
3	Модуль		2	20
всего			7	48

2. Определим трудозатраты на уровне композиции приложения, приняв среднюю производительность программиста 13 точек в месяц:

$$\text{Трудозатраты} = (\text{NOP} * (100 - \text{RUSE}) / 100) / \text{PROD} = (48 * (100 - 10) / 100) / 13 = \mathbf{3,32 \text{ (чел./мес.)}}$$



Модель ранней разработки архитектуры

Трудозатраты = $2,94 * EArch * (Размер)^p$,

где: Трудозатраты (работа) — число человеко-месяцев

$EArch = PERS * RCPX * RUSE * PDIF * PREX * FCIL * SCED$

Размер — KSLOC (предпочтительно для подсчета KSLOC предварительно подсчитать количество функциональных точек)

p — показатель степени

Время = $3,67 * (Трудозатраты)^{(0.28 + 0.2 * (p-1.01))}$

Примечание: Множитель EArch является произведением семи показателей, характеризующих проект и процесс создания ПО, а именно: надежность и уровень сложности разрабатываемой системы (RCPX), повторное использование компонентов (RUSE), сложность платформы разработки (PDIF), возможности персонала (PERS), опыт персонала (PREX), график работ (SCED) и средства поддержки (FCIL). Каждый множитель может быть оценен экспертно, либо его можно вычислить путем комбинирования значений более детализированных показателей, которые используются на постархитектурном уровне (см. след. слайд)



Модель ранней разработки архитектуры

Множитель трудоемкости	Идентификатор	Составные драйверы затрат
Надежность и сложность продукта	RCPX	RELY-DATA-CPLX-DOCU
Повторное использование компонентов	RUSE	RUSE
Сложность платформы	PDIF	TIME-STOR-PVOL
Опытность персонала	PERS	AEXP-PEXP-LTEX
Способности персонала	PREX	ACAP-PCAP-PCON
Возможности среды	FCIL	TOOL-SITE
Сроки	SCED	SCED

Примечание: высокий опыт персонала и интенсивное повторное использование компонентов ведут к снижению затрат



Значения множителей трудоемкости, в зависимости от их уровня

	Оценка уровня множителя трудоемкости						
	Сверхнизкий	Очень низкий	Низкий	Номинальный	Высокий	Очень высокий	Сверхвысокий
PERS	2,12	1,62	1,26	1,00	0,83	0,63	0,5
RCPX	0,49	0,60	0,83	1,00	1,33	1,91	2,72
RUSE	n/a	n/a	0,95	1,00	1,07	1,15	1,24
PDIF	n/a	n/a	0,87	1,00	1,29	1,81	2,61
PREX	1,59	1,33	1,22	1,00	0,87	0,73	0,62
FCIL	1,43	1,30	1,10	1,00	0,87	0,73	0,62
SCED	n/a	1,43	1,14	1,00	1,00	1,00	n/a



Комментарии к множителям трудоемкости

- ▶ **PERS** — квалификация персонала (Сверхнизкий — аналитики и программисты имеют низшую квалификацию, текучесть больше 45%; Сверхвысокий — аналитики и программисты имеют высшую квалификацию, текучесть меньше 4%)
- ▶ **RCPX** — сложность и надежность продукта (Сверхнизкий — продукт простой, специальных требований по надежности нет, БД маленькая, документация не требуется; Сверхвысокий — продукт очень сложный, требования по надежности жесткие, БД сверхбольшая, документация требуется в полном объеме)
- ▶ **RUSE** — разработка для повторного использования (Низкий — не требуется; Сверхвысокий — требуется переиспользование в других продуктах)
- ▶ **PDIF** — сложность платформы разработки (Сверхнизкий — специальные ограничения по памяти и быстродействию отсутствуют, платформа стабильна; Сверхвысокий — жесткие ограничения по памяти и быстродействию, платформа нестабильна)
- ▶ **PREX** — опыт персонала (Сверхнизкий — новое приложение, инструменты и платформа; Сверхвысокий — приложение, инструменты и платформа хорошо известны)
- ▶ **FCIL** — оборудование (Сверхнизкий — инструменты простейшие, коммуникации затруднены; Сверхвысокий — интегрированные средства поддержки жизненного цикла, интерактивные мультимедиа коммуникации)
- ▶ **SCED** — сжатие расписания (Очень низкий — 75% от номинальной длительности; Очень высокий — 160% от номинальной длительности)



Модель этапа постархитектуры

Трудозатраты = $2,45 * E_{App} * (Размер)^p$,

где:

Трудозатраты (работа) — число человеко-месяцев

E_{App} — результат применения семнадцати уточняющих факторов, применяемых в данной модели

Время = $3,67 * (Трудозатраты)^{(0.28 + 0.2 * (p-1.01))}$



Усовершенствованная постархитектурная модель COCOMO II

Идентификатор	Драйвер	Назначение	Пояснения
RELY	Требуемая надежность	Учитывает ущерб от ошибок : от простых неудобств для пользователя до катастрофических последствий (потеря человеческих жизней или огромные финансовые убытки)	Чем выше требования к надежности, тем больше времени требуется на тщательное проектирование, кодирование и на экстремально глубокое тестирование. Основной прирост трудоемкости при высоком RELY приходится на фазы верификации и валидации (V&V). Высокая надежность (Very High) увеличивает трудозатраты на 26% относительно номинала
DATA	Размер базы данных	Оценивает влияние объема данных на трудоемкость разработки. Он учитывает соотношение между размером базы данных (в байтах) и количеством строк исходного кода (LOC)	Для определения уровня драйвера используется показатель D/P (отношение данных к программе) Рост трудозатрат обусловлен необходимостью сложной нормализации, индексации и оптимизации запросов, вызванных большим объемом данных, высокой трудоемкостью задачи создания реалистичных тестовых наборов данных большого объема, а также проблемами администрирования



Усовершенствованная постархитектурная модель COCOMO II

Идентификатор	Драйвер	Назначение	Пояснения
CPLX *	Сложность продукта	Оценивает техническую сложность программного продукта. Данный драйвер определяет, насколько трудно реализовать алгоритмы, логику управления и структуры данных в проекте	Сложность разделена на 5 подобластей: а) операции управления: ветвления, циклы, логика; б) вычислительные операции: от простой арифметики до обработки сигналов и высшей математики; в) операции с данными: простые переменные vs динамические структуры и СУБД; г) управление ресурсами : взаимодействие с ОС, памятью, процессором; д) пользовательский интерфейс: текстовый ввод vs сложная 3D-графика или мультимедиа. Для каждой из них прописаны критерии, и итоговый рейтинг (от Very Low до Extra High) выбирается по самому высокому значению в любой из них

Примечание: * - новый параметр, который был добавлен в модели COCOMOII



Усовершенствованная постархитектурная модель COCOMO II

Идентификатор	Драйвер	Назначение	Пояснения
RUSE*	Требуемый уровень повторного использования	Оценивает дополнительные трудозатраты, необходимые для создания компонентов, которые планируется использовать в будущих проектах. RUSE — это про написание нового кода так, чтобы его можно было легко переиспользовать потом	Оценивается степень, в которой разрабатываемое ПО должно быть универсальным, документированным и протестированным для повторного использования в других программных продуктах компании, так как создание универсального компонента требует гораздо больше усилий, чем написание кода «под одну задачу»: нужно продумать обобщенные интерфейсы, написать более подробную документацию и провести более широкое тестирование
DOCU*	Документация	Оценивает объем и тщательность подготовки документации, необходимой для сопровождения жизненного цикла ПО. Этот драйвер определяет, насколько детально документируется проект относительно реальных нужд разработки и последующей эксплуатации	Оценивается соответствие объема проектной документации (ТЗ, архитектурные решения, руководства пользователя, регламенты тестирования) специфике проекта. Чем больше формальной документации требуется создавать «по регламенту», тем больше времени уходит у команды на её написание и актуализацию, что увеличивает общие трудозатраты



Усовершенствованная постархитектурная модель COCOMO II

Идентификатор	Драйвер	Назначение	Пояснения
TIME	Ограничение времени выполнения	Оценивает ограничения по времени выполнения программы на центральном процессоре. Данный драйвер отражает сложность разработки ПО в условиях дефицита вычислительных мощностей	Если программа должна работать на пределе возможностей процессора (например, в системах реального времени, видеоиграх или на бортовых компьютерах), разработчикам приходится тратить гораздо больше усилий на оптимизацию кода, настройку алгоритмов и ручное управление памятью. Extra High уровень драйвера на 63% увеличивает трудозатраты, что связано со сложностью поиска ошибок, возникающих только при пиковых нагрузках, невозможностью использования «тяжелых» библиотек или стандартных функций, если они слишком нагружают процессор и, как следствие, приходится специально писать низкоуровневые решения



Усовершенствованная постархитектурная модель COCOMO II

Идентификатор	Драйвер	Назначение	Пояснения
STOR	Ограничение объема основной памяти	Оценивает ограничения по объему оперативной памяти , доступной для работы программы. Данный драйвер отражает дополнительные трудозатраты, которые возникают в случае необходимости «встраивать» код в жесткие рамки памяти	Когда оперативной памяти достаточно, программисты используют стандартные библиотеки и простые структуры данных. Если же памяти не хватает (например, в микроконтроллерах, встраиваемых системах или высоконагруженных серверах), приходится тратить время на сложную оптимизацию: использование кэширования, сжатие данных, ручное управление памятью и сегментацию кода. Трудозатраты возрастают в связи с необходимостью использовать более сложные алгоритмы и структуры данных (например, вместо простых массивов применять динамические структуры с упаковкой данных). В условиях дефицита памяти даже малейшая ее «утечка» приводит к тяжёлым последствиям для системы, поэтому время на тестирование и отладку сильно возрастает. Часто приходится отказываться от удобных современных инструментов, которые потребляют много ресурсов, и писать всё «с нуля»



Усовершенствованная постархитектурная модель COCOMO II

Идентификатор	Драйвер	Назначение	Пояснения
PVOL*	Изменчивость платформы	Оценивает изменчивость программно-аппаратной платформы , на которой ведется разработка и будет работать продукт. Данный драйвер учитывает, насколько часто меняются операционная система, базы данных, сетевые протоколы или оборудование в ходе жизненного цикла проекта	«Платформой» считается: а) операционная система: обновления версий б) СУБД: изменения в движке или языке запросов; в) компиляторы и библиотеки: выход новых версий фреймворков, г) «железо»: если софт пишется под конкретное оборудование, которое еще проектируется. Если платформа нестабильна, разработчикам приходится постоянно переписывать части кода, резко увеличивает трудозатраты
ACAP	Способности аналитика	Оценивает профессиональные способности системных аналитиков , участвующих в проекте. Под аналитиками понимаются специалисты, отвечающие за разработку требований, спецификаций и проектирование верхнего уровня	Данный драйвер отражает уровень квалификации аналитиков в команде проекта: их способность: анализировать требования; проектировать архитектуру; выявлять и разрешать риски на ранних стадиях; формулировать чёткие спецификации для разработчиков. Чем выше уровень аналитиков, тем меньше времени и усилий потребуется на проектирование и исправление ошибок в будущем



Усовершенствованная постархитектурная модель

SOCOMO II

Идентификатор	Драйвер	Назначение	Пояснения
AEXP	Опыт работы с приложениями	Оценивает практический опыт команды аналитиков в конкретной предметной области проекта. Если драйвер ACAP оценивает врожденные способности и интеллект, то AEXP фокусируется на накопленном стаже работы в аналогичных проектах	Оценивается опыт аналитиков в работе с аналогичными приложениями, т.е. насколько они знакомы с предметной областью проекта, типами решаемых задач (например: учёт, аналитика, обработка транзакций), типовыми архитектурами и шаблонами проектирования для таких систем
PCAP	Способности программиста	Оценивает профессиональные способности программистов , работающих над проектом. При этом учитываются специалисты, которые пишут код и проводят модульное тестирование	Оценивается способность программистов к эффективному написанию кода, скорость работы, навыки отладки, понимание алгоритмов и общая техническая грамотность. Если команда состоит из супер-профессионалов программирования (уровень Very High), то трудозатраты будут на 24% меньше номинальных
PCON*	Преемственность персонала	Оценивает стабильность состава команды в ходе выполнения проекта, т.е. фактически – текучесть кадров	Оценивается ежегодный процент увольнений или ротаций персонала в организации. Чем чаще меняются люди в команде, тем больше времени тратится на обучение новых сотрудников и передачу знаний («вход в проект»), что снижает общую эффективность. Высокая текучесть кадров (уровни драйвера Low/Very Low) увеличивает трудозатраты. Стабильный коллектив (уровни High/Very High) позволяет завершить проект быстрее и дешевле

Усовершенствованная постархитектурная модель COCOMO II

Идентификатор	Драйвер	Назначение	Пояснения
LTEX*	Знание языка программирования и инструментария	Оценивает опыт команды в использовании конкретного языка программирования и инструментальных средств, выбранных для проекта	Оценивается уровень владения выбранными технологиями. Если команда уже работала с выбранным технологическим стеком (например, C++ + Qt), она пишет код быстрее и делает меньше ошибок. Если же технологии для команды новые, трудозатраты существенно возрастают из-за времени на обучение. При этом оценивается не только знание синтаксиса языка, но и владение библиотеками, средами разработки (IDE), системами сборки и тестирования
SITE*	Распределенная разработка. Взаимодействие между командами разработчиков	Оценивает степень территориальной распределенности команды и качество средств коммуникации между участниками проекта	Данный драйвер учитывает, что работа в одном офисе («лицом к лицу») всегда эффективнее, чем работа распределенных команд, из-за накладных расходов на связь, разницы во времени и наличия культурных барьеров. Например: ожидание ответа на вопрос по электронной почте может занять часы, в то время как коллега за соседним столом ответит за секунды или если команда находится в США, а разработчики в Индии, временное окно для живого общения составляет всего 1–2 часа в сутки



Усовершенствованная постархитектурная модель COCOMO II

Идентификатор	Драйвер	Назначение	Пояснения
РЕХР	Опыт работы с платформой	Оценивает опыт проектной команды в работе с конкретной вычислительной платформой , которая будет использоваться в проекте, т.е. аппаратным и программным окружением, на котором будет функционировать ПО. Под «платформой» понимается совокупность операционной системы, графического интерфейса, базы данных и сетевых протоколов (например, разработка под iOS, Android, облачную инфраструктуру и пр.)	Оценивается насколько хорошо разработчики знают особенности и ограничения конкретной программно-аппаратной среды. Если команда уже работала с данной платформой, она не тратит время на изучение документации, что снижает трудозатраты

Примечание: В COCOMO II опыт разделен на три разных драйвера, чтобы точнее настроить модель:

- АЕХР — знание предметной области (бизнес-логика)
- РЕХР — знание платформы (ОС, СУБД, промежуточное ПО)
- LTEХ — знание языка программирования и инструментов



Усовершенствованная постархитектурная модель COCOMO II

Идентификатор	Драйвер	Назначение	Пояснения
TOOL	Использование программных инструментов	Оценивает степень автоматизации разработки и использование современных программных инструментов (IDE, CASE-средства, системы контроля версий, инструменты тестирования и т.д.). Т.е. акцент делается на технической оснащенности рабочих мест членов команды	Оценивается насколько интегрированы и современны инструменты, которые использует команда на всех этапах жизненного цикла создания ПО. Использование мощных современных инструментов (уровень Very High) значительно сокращает рутинную работу и уменьшает итоговую трудоемкость
SCED	Требуемые сроки разработки	Отвечает за сжатие или растягивание графика разработки относительно номинального срока	Если проект нужно завершить быстрее, чем предполагает расчетная норма, трудозатраты возрастают. Это происходит из-за увеличения расходов на коммуникацию между большим количеством сотрудников и неизбежных ошибок при спешке. Считается практически невозможным сжать график более чем до 75% от номинального без катастрофической потери качества или развала проекта («закон Брукса»)



Драйверы затрат в модели COSOMO II

Идентификатор	Уточняющий фактор работ	Диапазон изменения параметра	Очень низкий	Низкий	Номинальный	Высокий	Очень высокий	Чрезвычайно высокий
RELY	Требуемая надежность	0.81 – 1.26	0.81	0.91	1.0	1.1	1.26	
DATA	Размер базы данных	0.9 – 1.28		0.9	1.0	1.14	1.28	
CPLX	Сложность продукта	0.73 – 1.74	0.73	0.87	1.0	1.17	1.34	1.74
RUSE	Требуемый уровень повторного использования	0.98 – 1.34		0.98	1.0	1.07	1.15	1.34
DOCU	Соответствие документации требованиям жизненного цикла	0.81 – 1.23	0.81	0.91	1.0	1.11	1.23	
TIME	Ограничение времени выполнения	1.0 – 1.63			1.0	1.11	1.29	1.63
STOR	Ограничение объема основной памяти	1.0 – 1.46			1.0	1.05	1.17	1.46
PVOL	Изменчивость платформы	0.87 – 1.3		0.87	1.0	1.15	1.3	
ACAP	Способности аналитика	0.71 – 1.42	1.42	1.19	1.0	0.85	0.71	
AEXP	Знание приложений	0.81 – 1.22	1.22	1.1	1.0	0.88	0.81	
PCON	Преимственность персонала	0.81 – 1.29	1.29	1.12	1.0	0.9	0.81	
PCAP	Способности программиста	0.76 – 1.34	1.34	1.15	1.0	0.88	0.76	
PEXP	Опыт работы с платформой	0.85 – 1.19	1.19	1.09	1.0	0.91	0.85	
LTEX	Знание языка программирования и инструментария	0.84 – 1.2	1.2	1.09	1.0	0.91	0.84	
TOOL	Использование программных инструментов	0.78 – 1.17	1.17	1.09	1.0	0.9	0.78	
SITE	Распределенная разработка	0.8 – 1.22	1.22	1.09	1.0	0.93	0.86	0.8
SCED	Требуемые сроки разработки	0.75 – 1.6	0.75	0.85	1.0	1.3	1.6	

Краткое пояснение к драйверам затрат

	Драйвер	Очень низкий	Низкий	Номинальный	Высокий	Очень высокий	Чрезвычайно высокий
RELY	<i>Требуемая надежность</i>	Незначительные неудобства	Небольшие потери, легко восстанавливаемые данные	Умеренные потери, но данные восстанавливаемы	Значительные финансовые потери или ущерб имиджу	Риск для человеческой жизни	
DATA	<i>Размер базы данных</i>		$D/P < 10$	$10 \leq D/P < 100$	$100 \leq D/P < 1000$	$D/P \geq 1000$	
CPLX	<i>Сложность продукта</i>	Простые CRUD-приложения, прямая логика	Типовой бизнес-софт, простые скрипты	Средний уровень сложности (стандарт для большинства систем)	Сложные алгоритмы, многопоточность, сетевые протоколы	Системы реального времени, криптография, ОС	Микрокод, сложная обработка сигналов, ИИ-движки
RUSE	<i>Требуемый уровень повторного использования</i>		Повторного использования не требуется	Переиспользование в рамках одной программы / проекта	Переиспользование в рамках всей продуктовой линейки	Переиспользование в разных проектах компании	Создание публичных библиотек или внешних стандартов



Оценки сложности (CPLX) в зависимости от типа модуля

	Операции по управлению	Вычислительные операции	Аппаратно-зависимые операции	Операции по управлению данными	Операции по управлению пользовательским интерфейсом
Очень низкий	Линейный код с несколькими не вложенными операторами структурированного программирования. Простая компоновка модуля с помощью вызовов подпрограмм или простых скриптов	Вычисление простых выражений: например, $A=B+C*(DE)$	Простое чтение, запись инструкций в простых форматах	Простые массивы в основной памяти. Простые запросы и обновления БД	Простые формы ввода, генератор отчетов
Низкий	Простое вложение операторов структурного программирования. В основном простые предикаты	Вычисление выражений среднего уровня: например, $d = \sqrt{b^2 - 4 * a * c}$	Не требуется знаний характеристик конкретного процессора или устройства ввода-вывода. Ввод-вывод выполняется на уровне GET/PUT	Единичный файл без изменений структуры данных, без правок, без промежуточных файлов. Умеренно сложные запросы и обновления БД	Использование простого графического пользовательского интерфейса (GUI)



Оценки сложности в зависимости от типа модуля

	Операции по управлению	Вычислительные операции	Аппаратно-зависимые операции	Операции по управлению данными	Операции по управлению пользовательским интерфейсом
Номинальный	В основном простая вложенность. Некоторый межмодульный контроль. Простые вызовы или передача сообщений, включая распределенную обработку	Использование стандартных математических и статистических функций. Основные матрично-векторные операции	Обработка ввода-вывода включает в себя выбор устройства, проверку состояния и обработку ошибок	Многофайловый ввод и однофайловый вывод. Простые структурные изменения, простые правки. Сложные запросы к базе данных и обновления	Использование простого набора виджетов
Высокий	Сильно вложенные операторы структурного программирования со множеством составных предикатов. Однородная, распределенная обработка. Однопроцессорное программное управление в режиме реального времени	Базовый численный анализ: многомерная интерполяция, обыкновенные дифференциальные уравнения	Операции на физическом уровне ввода -вывода (преобразование адресов физического хранилища; поиск, считывание). Оптимизированное перекрытие ввода-вывода	Простые триггеры, активируемые содержимым потока данных. Сложная реструктуризация данных	Разработка и расширение набора виджетов. Простой голосовой ввод-вывод, мультимедиа



Оценки сложности в зависимости от типа модуля

	Операции по управлению	Вычислительные операции	Аппаратно-зависимые операции	Операции по управлению данными	Операции по управлению пользовательским интерфейсом
Очень высокий	Реентерабельное и рекурсивное кодирование. Обработка прерываний с фиксированным приоритетом. Синхронизация задач, сложные обратные вызовы, гетерогенная распределенная обработка. Однопроцессорное жесткое управление в режиме реального времени.	Сложный, но структурированный численный анализ: почти сингулярные матричные уравнения, уравнения в частных производных. Простое распараллеливание	Процедуры для диагностики прерываний, обслуживания, маскировки. Встраиваемые системы с высокой производительностью	Координация распределенной базы данных. Сложные триггеры. Оптимизация поиска	Умеренно сложная 2D / 3D динамическая графика, мультимедиа
Чрезвычайно высокий	Планирование нескольких ресурсов с динамически меняющимися приоритетами. Управление на уровне микрокода. Распределенный жесткий контроль в режиме реального времени	Сложный и неструктурированный численный анализ: высокоточный анализ зашумленных, стохастических данных. Сложное распараллеливание	Кодирование, зависящее от времени работы устройства, микропрограммируемые операции. Встраиваемые системы, критически важные для производительности	Высоко-связные, динамичные реляционные и объектные структуры. Управление данными на естественном языке	Сложные мультимедиа, виртуальная реальность



Краткое пояснение к драйверам затрат

	Драйвер	Очень низкий	Низкий	Номинальный	Высокий	Очень высокий	Чрезвычайно высокий
DOCU	<i>Соответствие документации требованиям жизненного цикла</i>	Документация практически отсутствует	Минимальный набор документов (черновики)	Стандартный набор для жизненного цикла	Повышенные требования (ГОСТы, стандарты)	Избыточная, крайне детальная документация	
TIME	<i>Ограничение времени выполнения</i>			50% использования доступного времени процессора (ресурсов процессора хватает с запасом)	70% (заметная нагрузка)	85% (серьезная оптимизация)	95% (критическая нагрузка)
STOR	<i>Ограничение объема основной памяти</i>			50% использования доступной оперативной памяти (памяти хватает с запасом)	70% (заметные ограничения)	85% (серьезная экономия)	95% (критический дефицит)
PVOL	<i>Изменчивость платформы</i>		Изменения каждые 12 месяцев (стабильная среда)	Изменения каждые 6 месяцев	Изменения каждые 2 месяца	Изменения каждые 2 недели (очень нестабильная среда)	



Краткое пояснение к драйверам затрат

	Драйвер	Очень низкий	Низкий	Номинальный	Высокий	Очень высокий
АСАР	Способности аналитика	Процентиль способностей 15	35 проц.	55 проц.	75 проц.	90 проц.

Процентиль способностей (percentile rank) — это статистический показатель, который показывает, какой процент людей из контрольной (нормативной) группы показал результаты **хуже**, чем конкретный тестируемый

Параметр АСАР	
Очень низкий	Аналитики с минимальными навыками. Слабое понимание предметной области и методов анализа. Требуется постоянное руководство
Низкий	Ограниченные навыки анализа. Могут выполнять базовые задачи, но допускают ошибки в интерпретации требований
Номинальный	Базовые навыки анализа требований и проектирования. Способны работать самостоятельно
Высокий	Опытные аналитики с хорошими навыками. Способны выявлять скрытые требования, риски и предлагать оптимальные решения
Очень высокий	Глубокое понимание предметной области, передовые методы анализа (например, моделирование бизнес-процессов, UML, BPMN)



Краткое пояснение к драйверам затрат

	Драйвер	Очень низкий	Низкий	Номинальный	Высокий	Очень высокий
АЕХР	Знание приложений	Стаж в предметной области ≤ 2 мес.	6 мес.	1 год	3 года	6 лет

Параметр АЕХР	
Очень низкий	Команда аналитиков не имеет опыта работы с подобными приложениями. Требуется глубокое погружение в предметную область
Низкий	Ограниченный опыт. Есть частичное понимание типовых решений, но требуются дополнительные исследования и консультации
Номинальный	Средний уровень, типичный для отрасли. Базовые знания предметной области и типовых архитектур. Команда может работать самостоятельно
Высокий	Аналитики имеют опыт работы с аналогичными системами. Знают типовые проблемы и решения, может предложить оптимальные подходы
Очень высокий	Эксперты с многолетним опытом в данной предметной области. Обладают глубоким пониманием бизнес-процессов и технических нюансов. Могут обучать других



Краткое пояснение к драйверам затрат

	Драйвер	Очень низкий	Низкий	Номинальный	Высокий	Очень высокий
PCON	Преемственность персонала	Процент текучести в год 48%	24% в год	12% в год	6% в год	3% в год

Параметр PCON	
Очень низкий	Высокая текучесть. Ожидается, что более 50% команды покинут проект до его завершения. Частые замены ключевых ролей
Низкий	Значительная текучесть. Прогнозируется уход 30–50% участников. Потребуется замена нескольких ключевых специалистов
Номинальный	Умеренная текучесть, типичная для отрасли. Возможен уход до 20% команды. Основные роли сохраняются
Высокий	Низкая текучесть. Ожидается уход менее 10% участников. Ключевые специалисты остаются до конца проекта
Очень высокий	Состав команды практически неизменен. Все ключевые участники гарантированно работают до завершения проекта



Краткое пояснение к драйверам затрат

	Драйвер	Очень низкий	Низкий	Номинальный	Высокий	Очень высокий
PCAP	Способности программиста	Процентиль способностей - нижние 15%	Нижние 35%	Средние 55%	Верхние 75%	Верхние 90%

Параметр PCAP

Очень низкий	Ограниченные навыки, частые ошибки, низкая производительность. Требуют постоянного наставничества
Низкий	Программисты с базовыми навыками. Могут выполнять типовые задачи, но испытывают трудности с нестандартными проблемами
Номинальный	Средний уровень, типичный для отрасли. Способны самостоятельно решать большинство задач. Имеют базовые знания лучших практик
Высокий	Быстро решают сложные задачи, пишут эффективный код, применяют передовые методы
Очень высокий	Глубокие знания технологий, способность проектировать архитектуры. Высокая производительность

Примечания: Процентиль способностей показывает, какой процент программистов в контрольной (нормативной) группе показал результаты хуже, чем данная команда. Например, 75-й процентиль означает, что команда превосходит 75 % других команд по уровню квалификации

Оценка драйвера PCAP основывается на способностях программистов как команды, а не отдельных людей



Краткое пояснение к драйверам затрат

	Драйвер	Очень низкий	Низкий	Номинальный	Высокий	Очень высокий
РЕХР	Опыт работы с платформой	Стаж работы с платформой ≤ 2 мес.	6 мес.	1 год	3 года	6 лет

Параметр РЕХР

Очень низкий	Команда не имеет опыта работы с данной платформой. Требуется глубокое погружение в особенности окружения
Низкий	Имеется ограниченный опыт. Есть частичное понимание типовых решений, но требуются дополнительные исследования и консультации
Номинальный	Команда имеет базовые знания платформы и типовых архитектур и может работать самостоятельно
Высокий	Команда имеет опыт работы с аналогичной платформой. Знает типовые проблемы и решения, может предложить оптимальные подходы к развёртыванию и настройке
Очень высокий	Глубокое понимание технических нюансов, способность оптимизировать производительность и решать сложные инфраструктурные задачи



Краткое пояснение к драйверам затрат

	Драйвер	Очень низкий	Низкий	Номинальный	Высокий	Очень высокий
LTEX	Знание языка программирования и инструментария	Опыт работы с технологическим стеком ≤ 2 мес.	6 мес.	1 год	3 года	6 лет

Параметр LTEX	
Очень низкий	Команда не имеет опыта работы с данными языками и инструментами. Требуется глубокое погружение в технологии
Низкий	Ограниченный опыт. Есть частичное понимание базовых возможностей, но требуются дополнительные исследования и консультации
Номинальный	Средний уровень, типичный для отрасли. Базовые знания языков и инструментов. Команда может работать самостоятельно
Высокий	Команда имеет опыт работы с аналогичными технологиями. Знает типовые проблемы и решения, может предложить оптимальные подходы к использованию инструментов
Очень высокий	Команда обладает многолетним опытом в работе с данными языками и инструментами, глубоким пониманием нюансов и способностью оптимизировать процессы. Могут обучать других



Краткое пояснение к драйверам затрат

	Драйвер	Очень низкий	Низкий	Номинальный	Высокий	Очень высокий
TOOL	<i>Использование программных инструментов</i>	Используются только базовые инструменты — текстовые редакторы, компиляторы командной строки. Никакой интеграции между ними нет	Простые CASE-средства, редакторы с подсветкой синтаксиса, отдельные отладчики	Стандартные современные IDE с базовой поддержкой жизненного цикла разработки	Интегрированные системы управления жизненным циклом (ALM), средства автоматического тестирования и управления конфигурациями	Полностью интегрированные программные комплексы, поддерживающие проектирование, генерацию кода, формальную верификацию и управление требованиями
SITE	<i>Распределенная разработка</i>	См. следующий слайд				
SCED	<i>Требуемые сроки разработки</i>	75% от номинального	85% от номинального	100% от номинального	130% от номинального	160% от номинального



Учет влияния распределенной разработки и коммуникационной поддержки (драйвер SITE)

	Драйвер	Очень низкий	Низкий	Номинальный	Высокий	Очень высокий	Чрезвычайно высокий
SITE	<i>Распределенная разработка (колокация)</i>	Международные площадки, разные часовые пояса	Разные города или регионы	Один город или область	Один комплекс зданий или кампус	Одна площадка, один офис	Виртуальная «общая комната»
SITE	<i>Распределенная разработка (коммуникация)</i>	Почта, редкие телефонные звонки	Телефон, факс, электронная почта	Электронная почта, регулярные телефонные конференции	Широкополосная связь, видео-конференции по запросу	Интерактивные мультимедиа, мгновенная связь, общие рабочие пространства	VR, продвинутые доски



Правила вычисления показателя степени в модели СОСОМО II

- ▶ Значение показателя степени изменяется от 1.1 до 1.24
- ▶ Значение показателя степени зависит от того, насколько новаторским является данный проект, от гибкости процесса разработки ПО, от применяемых процессов управления рисками, сплоченности команды программистов и уровня управления организацией-разработчиком
- ▶ Значение показателя степени рассчитывается с учетом пяти показателей по восьми балльной шкале от низшего (7 баллов) до наивысшего (0 баллов) уровня
- ▶ Значения всех пяти показателей суммируются, сумма делится на 100, результат прибавляется к числу 1.01



Факторы, влияющие на показатель степени в модели СОСОМО II

- ▶ Наличие прецедентов у приложения: уровень опыта организации-разработчика в данной области
- ▶ Гибкость процесса: степень строгости контракта, порядок его выполнения, присущая контракту свобода внесения изменений, виды деятельности в течение жизненного цикла и взаимодействие между заинтересованными сторонами
- ▶ Проработанность архитектуры и разрешение связанных с ней рисков: степень технической осуществимости, продемонстрированной до перехода к полномасштабному производству
- ▶ Сплоченность команды: степень сотрудничества и того, насколько все заинтересованные стороны (покупатели, разработчики, пользователи, ответственные за сопровождение и др.) разделяют общую концепцию
- ▶ Зрелость процесса: уровень зрелости организации-разработчика, определяемая в соответствии с моделью СММ



Новизна проекта (PREC)

Отражает предыдущий опыт организации в реализации проектов данного типа. Очень низкий уровень этого показателя означает отсутствие опыта, наивысший уровень указывает на компетентность организации-разработчика в данной области ПО

Характеристика	Рекомендуемое значение
Полное отсутствие прецедентов, полностью непредсказуемый проект	6,2
Почти полное отсутствие прецедентов, в значительной мере непредсказуемый проект	4,96
Наличие некоторого количества прецедентов	3,72
Общее знакомство с проектом	2,48
Значительное знакомство с проектом	1,24
Полное знакомство с проектом	0



Гибкость процесса разработки (FLEX)

Отображает возможность изменения процесса разработки ПО. Очень низкий уровень этого показателя означает, что процесс определен заказчиком заранее, наивысший — заказчик определил лишь общие задачи без указания конкретной технологии процесса разработки ПО

Характеристика	Рекомендуемое значение
Точный, строгий процесс разработки	5,07
Случайные послабления в процессе	4,05
Некоторые послабления в процессе	3,04
Большей частью согласованный процесс	2,03
Некоторое согласование процесса	1,01
Заказчик определил только общие цели	0



Разрешение рисков в архитектуре системы (RESL)

Отображает степень детализации анализа рисков, основанного на анализе архитектуры системы. Очень низкий уровень данного показателя соответствует поверхностному анализу рисков, наивысший уровень означает, что был проведен тщательный и полный анализ всевозможных рисков

Характеристика	Рекомендуемое значение
Малое (20 %)	7
Некоторое (40 %)	5,65
Частое (60 %)	4,24
В целом (75 %)	2,83
Почти полное (90 %)	1,41
Полное (100%)	0



Сплоченность команды (TEAM)

Отображает степень сплоченности команды и их способность работать совместно. Очень низкий уровень этого показателя означает, что взаимоотношения в команде сложные, а наивысший — что команда сплоченная и эффективная в работе, не имеет проблем во взаимоотношениях

Характеристика	Рекомендуемое значение
Сильно затрудненное взаимодействие	5,48
Несколько затрудненное взаимодействие	4,38
Некоторая согласованность	3,29
Повышенная согласованность	2,19
Высокая согласованность	1,1
Взаимодействие как в едином целом	0



Уровень зрелости процесса разработки (РМАТ)

Отображает уровень развития процесса создания ПО в организации-разработчике

Характеристика	Рекомендуемое значение
Уровень 1 CMM	7,8
Уровень 1+ CMM	6,24
Уровень 2 CMM	4,68
Уровень 3 CMM	3,12
Уровень 4 CMM	1,56
Уровень 5 CMM	0



Пример использования модели COSOMO II

Предположим, что организация-разработчик выполняет программный проект в той области, в которой у нее мало опыта разработок. Заказчик не определил строгий технологический процесс, который будет использоваться при создании программного продукта, но некоторые согласования были проведены. Заказчик не выделил в плане работ времени на анализ возможных рисков в архитектуре системы. Для создания программной системы необходимо сформировать новую команду специалистов. Организация-разработчик недавно привела в действие программу усовершенствования технологического процесса разработки ПО и может примерно котироваться как организация второго уровня в соответствии с моделью оценки уровня зрелости.

Предварительный размер проекта оценен в **28 KSLOC**.

Предполагается очень высокая надежность системы и сложность системных модулей, высокие ограничения объема памяти, низкий уровень использования программных инструментов и ускоренный график работы. Остальные драйверы затрат номинальные



Показатели проекта

Показатель проекта	Характеристика показателя	Значение
Новизна проекта	Это новый проект для организации, но предметная область является знакомой; данный показатель имеет низкий уровень	3,72
Гибкость процесса разработки	Некоторое согласование процесса разработки с заказчиком дает высокое значение показателю	1,01
Анализ архитектуры системы и рисков	Анализ не был проведен — уровень данного показателя очень низкий	7
Сплоченность команды	Команда разработчиков новая, информация о ней отсутствует - уровень этого показателя оценивается как обычный	3,29
Уровень развития процесса разработки	Организация имеет второй уровень зрелости процессов разработки	4,68
Итого		19,7

Значение показателя степени $p = 19,7/100 + 1,01 = 1,207$



Выполнение расчетов

$$\text{Трудозатраты} = 2,45 * E_{\text{App}} * (\text{Размер})^p$$

$$E_{\text{App}} = \text{RELY} * \text{CPLX} * \text{STOR} * \text{TOOL} * \text{SCED} = 1,26 * 1,34 * 1,05 * 1,09 * 0,75 = 1,45$$

$$\text{Трудозатраты} = 2,45 * 1,45 * 28^{1,207} = \mathbf{198} \text{ чел.-мес.}$$

$$\text{Время} = 3,67 * (\text{Трудозатраты})^{(0,28 + 0,2 * (p - 1,01))} = 3,67 * 198^{(0,28 + 0,2 * (1,207 - 1,01))} = \mathbf{19,9} \text{ мес.}$$



Достоинства модели СОСОМО II

- ▶ возможен учет достаточно полной номенклатуры факторов, влияющих на экономические характеристики производства сложных программных продуктов
- ▶ метод является достаточно универсальным и может поддерживать различные размеры, режимы и уровни качества продуктов
- ▶ фактические данные подбираются в соответствии с реальными проектами и факторами корректировки, которые могут соответствовать конкретному проекту и организации
- ▶ прогнозы производственных процессов являются варьируемыми и повторяемыми
- ▶ метод позволяет добавлять уникальные факторы для корректировки экономических характеристик, связанные со специфическим проектом и организацией
- ▶ возможна высокая степень достоверности калибровки с опорой на предыдущий опыт коллектива специалистов
- ▶ результаты прогнозирования сопровождаются обязательной документацией
- ▶ модель относительно проста в освоении и применении



Недостатки модели COSOMO II

- ▶ все результаты зависят от размера программного продукта: точность оценки размера, оказывает определяющее влияние на точность прогноза трудозатрат, длительности разработки и численности специалистов
- ▶ игнорируются требования к характеристикам качества программного продукта
- ▶ не учитывается зависимость между интегральными затратами и количеством времени, затрачиваемым на каждом этапе проекта
- ▶ игнорируется изменяемость требований к программному продукту в процессе производства
- ▶ недостаточно учитывается внешняя среда производства и применения программного продукта
- ▶ игнорируются многие особенности, связанные с аппаратным обеспечением проекта

Модель COSOMO II изначально представляет трудозатраты по сумме всех этапов производства (от этапа планирования концепции до этапа поставки программного продукта). При этом не учитываются проблемы сопровождения, переноса и повторного использования компонентов, которые трудно описывать в рамках одной и той же модели



Последовательное уточнение технико-экономических параметров программного проекта

1. Экспертные оценки экономических характеристик по прототипам – достоверность оценивания размера продукта 40 – 50%

2. Этап предварительного проектирования

- ▶ оценивание экономических характеристик в базовой модели COSOMO – достоверность оценивания размера продукта 20 – 30%
- ▶ оценивание экономических характеристик в детализированной модели COSOMO – достоверность оценивания размера продукта 10 – 20%

3. Этап детального проектирования

- ▶ оценивание экономических характеристик в упрощенной предварительной модели COSOMO достоверность оценивания размера продукта 5 – 10% (учитывается 7 параметров)
- ▶ оценивание экономических характеристик в детальной модели COSOMO II – достоверность оценивания размера продукта 3 – 5% (учитывается 17 параметров)



Экономика программной инженерии

Барышникова Марина Юрьевна
МГТУ им. Н.Э. Баумана
Каф. ИУ-7

baryshnikovam@mail.ru

Лекция 8

Оценка размера программного продукта на основе метода функциональных точек

Хорошей считается оценка, которая обеспечивает достаточно ясное представление реального состояния проекта и позволяет руководителю проекта принимать хорошие решения относительно того, как управлять проектом для достижения целей

Стив Макконнелл



Функционально-ориентированные метрики измерения программного продукта

- ▶ Функционально-ориентированные метрики косвенно измеряют программный продукт и процесс его разработки
- ▶ При этом рассматривается не размер, а функциональность или полезность продукта с точки зрения пользователя
- ▶ В качестве количественной характеристики применяется понятие количества функциональных точек FP (function points)

Будущая система описывается с использованием методологии структурного анализа и проектирования как многоуровневая графическая модель, представленная в виде иерархической совокупности взаимосвязанных функциональных диаграмм пользовательских бизнес-процессов (IDEF0-диаграмм). Каждый из бизнес-процессов включает в себя входные и выходные данные, преобразования и внешние интерфейсы. Декомпозиция бизнес-процессов производится до некоторой элементарной функции, реализация которой возможна в виде отдельного программного компонента. При декомпозиции следует учитывать все логические преобразования с данными



Метод функциональных точек

Функциональная точка - это единица измерения, выражающая объем бизнес-функциональности, которую программная система предоставляет пользователю

Функциональность программы связана с обработкой информации по запросу пользователя и не зависит от применяемых технических решений. Пользователи — это отправители и целевые получатели данных, ими могут быть как реальные люди, так и смежные интегрированные программно-технические системы

Определение числа функциональных точек является методом количественной оценки размера и сложности программной системы с точки зрения функций, которые система предоставляет пользователю

Метод функциональных точек позволяет:

- ▶ оценивать категории пользовательских бизнес-функций
- ▶ определять количество и сложность входных и выходных данных, их структуру, а также внешние интерфейсы, которые связаны с программной системой
- ▶ преодолеть трудности получения LOC – оценок на ранних стадиях жизненного цикла



Метод функциональных точек: историческая справка

Функциональные точки (function points) были впервые предложены в качестве метрики измерения размера программного продукта сотрудником компании IBM Аланом Альбрехтом (Allan Albrecht) в 1979 г. Постепенно метод функциональных точек превратился в индустриальный стандарт, и в 1986 г. для его поддержки и развития была создана некоммерческая организация IFPUG (International Function Point User Group)

Причины популярности метода

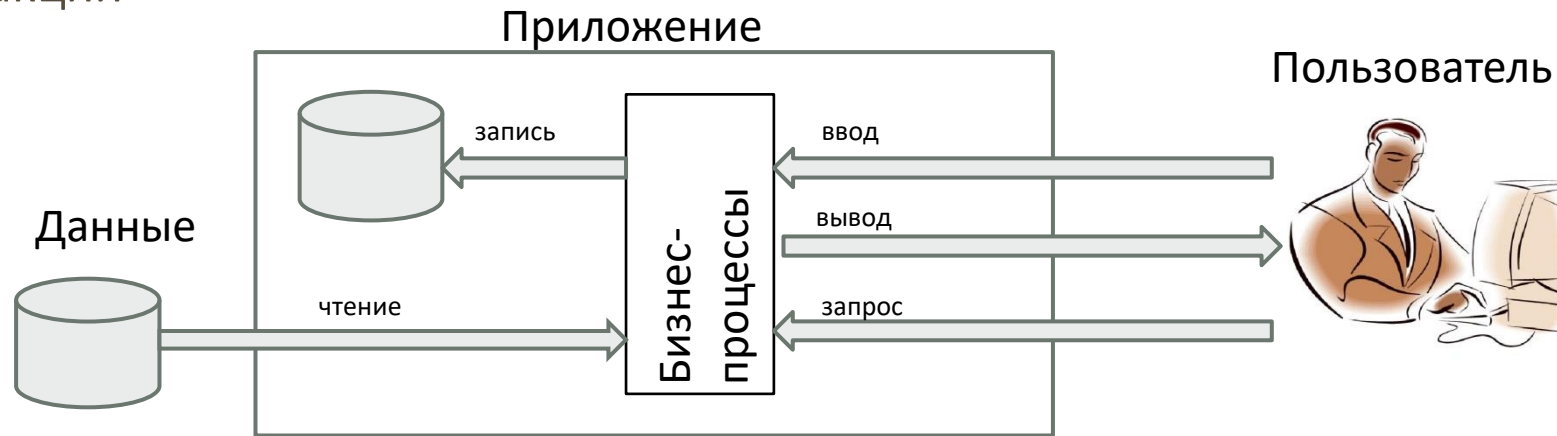
- ▶ Независимость от технологий: результаты не зависят от языка программирования, платформы или инструментов разработки
 - ▶ Ориентация на пользователя: оценка ведётся с позиции того, какие функции видит и использует конечный потребитель
 - ▶ Большая объективность оценок: Применение функциональных точек основано на изучении требований, поэтому оценка необходимых трудозатрат может быть выполнена на самых ранних стадиях работы над проектом и далее будет уточняться по ходу жизненного цикла, при этом чёткие правила подсчёта снижают субъективность оценок
-



Методика оценки трудоемкости разработки ПО на основе функциональных точек

Так как функциональные точки являются мерой оценки функциональности, т.е. полезности программного средства (ПС) с позиции пользователя, то общая функциональность измеряется путем:

- ▶ анализа логических групп данных, которые используются и поддерживаются приложением и характеризуют функциональность данных
- ▶ анализа вводимой и выводимой пользователем информации, т.е. функциональности совершаемых транзакций



Функциональность программного средства = Функциональность данных + Функциональность транзакций

Модель приложения для измерения функциональных точек



1. Измерение функциональности следует начинать с определения границ ПС, для чего надо ответить на вопросы:

- ▶ где располагается «граница системы», через которую проходят транзакции передаваемые или принимаемые продуктом, с точки зрения пользователя
- ▶ какие данные поддерживаются системой, а какие являются внешними по отношению к ней

Функции данных состоят из внутренних и внешних ресурсов, которые влияют на систему. Среди них следует выделять **внутренние логические файлы (ILF)** и **внешние интерфейсные файлы (EIF)**

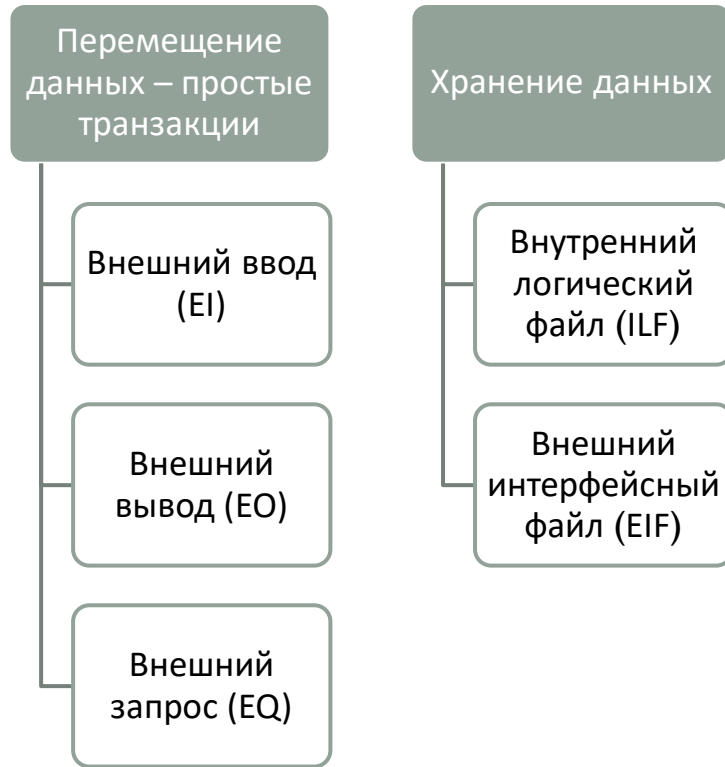
2. На следующем шаге необходимо определить элементарные процессы (наименьшие единицы функциональных требований), которые:

- ▶ имеют значение для пользователя
- ▶ представляют собой законченную транзакцию, оставляющую оцениваемое приложение в согласованном состоянии

Функции транзакции состоят из процессов, которыми измеряемое приложение обменивается с пользователем и другими внешними приложениями. К ним относятся **внешние вводы (EI)**, **внешние выводы (EO)** и **внешние запросы (EQ)**



Функциональные типы данных и типы транзакций, используемые в методе функциональных точек



Транзакция — это элементарный процесс, различаемый пользователем и перемещающий данные между внешней средой и программным приложением. В своей работе транзакции используют внутренние и внешние файлы

Внешний ввод (EI, транзакция, получающая данные от пользователя) — элементарный процесс, перемещающий данные из внешней среды в приложение. Данные могут поступать с экрана ввода или из другого приложения. Данные могут содержать как управляющую, так и деловую информацию. Обрабатываемые данные могут соответствовать одному или нескольким внутренним логическим файлам

Внешний вывод (EO, транзакция передающая данные пользователю) — элементарный процесс, перемещающий данные, вычисленные в приложении, во внешнюю среду и связанный с созданием и/или обработкой выходной информации приложения — выходного отчета, документа, экранной формы

Внешний запрос (EQ, интерактивный диалог с пользователем, требующий от него каких-либо действий) — элементарный процесс, состоящий из комбинации «запрос/ответ», не связанный с вычислением производных данных или обновлением внутренних логических файлов (базы данных)

Внутренний логический файл (ILF, информация, которая используется во внутренних взаимодействиях системы) — выделяемые пользователем логически связанные группы данных или блоки управляющей информации, которые поддерживаются внутри продукта и обслуживаются через внешние вводы

Внешний интерфейсный файл (EIF, файлы, участвующие во внешних взаимодействиях с другими системами) — выделяемые пользователем логически связанные группы данных или блоки управляющей информации, на которые ссылается продукт, но которые поддерживаются вне продукта

Источники данных

Тип транзакции	Элементы данных
Внешние вводы (элементарные процессы, в которых данные пересекают границу ПС снаружи внутрь)	Необходимо учитывать вводы с различных внешних устройств, органов управления, датчиков и т.п. в системах реального времени, а также дополнительные вводы из других приложений. Такие вводы должны обновлять внутренние логические файлы рассматриваемого ПС
Внешние выводы (элементарные процессы, в которых данные пересекают границу ПС изнутри наружу)	Файлы, продуцируемые в данном бизнес-процессе для передачи другим бизнес-процессам либо за пределы программной системы; единицы деловой информации, предназначенные для конечных пользователей, оформленные в виде экранных форм либо бумажных документов. Например, поля данных в отчетах, вычисляемые значения, сообщения об ошибках
Внешние запросы (элементарные процессы, в которых данные пересекают границу ПС в обе стороны в ходе диалога)	Источниками информации для определения внешних запросов являются форматы экранов и диалогов ввода-вывода, а также любые другие формы ввода-вывода данных в ПС, осуществляемые в диалоговом режиме. Вводимые элементы: поле, используемое для поиска, щелчок мыши. Выводимые элементы — отображаемые на экране поля



Внутренние и внешние файлы

- ▶ Внутренние логические файлы (ILF) и внешние интерфейсные файлы (EIF) являются логически связанными группами данных, которые представлены в виде файлов, таблиц баз данных и пр.
- ▶ Оба эти типа файлов могут использоваться приложением при генерации внешних выводов (EO), а также при обслуживании внешних запросов (EQ). Разница между ними состоит лишь в том, что ILF поддерживаются самим рассматриваемым ПС путем использования внешних вводов (EI), а EIF поддерживаются каким-либо другим ПС
- ▶ Внешние интерфейсные файлы обычно используются в пределах оцениваемой системы только для справочных целей



Детализация показателей оценки



Информационные характеристики, используемые в методе функциональных точек

- ▶ *Количество внешних вводов.* Подсчитываются все вводы пользователя, по которым поступают разные прикладные данные, при этом вводы должны быть отделены от запросов, которые подсчитываются отдельно
- ▶ *Количество внешних выводов.* Подсчитываются все выводы, по которым к пользователю поступают результаты, вычисленные программным приложением. В этом контексте выводы означают отчеты, экраны, распечатки, сообщения об ошибках. Индивидуальные единицы данных внутри отчета отдельно не подсчитываются
- ▶ *Количество внешних запросов.* Под запросом понимается диалоговый ввод, который приводит к немедленному программному ответу в форме диалогового вывода. При этом диалоговый ввод в приложении не сохраняется, а диалоговый вывод не требует выполнения вычислений. Подсчитываются все запросы — каждый учитывается отдельно
- ▶ *Количество внутренних логических файлов.* Подсчитываются все логические файлы (то есть логические группы данных, которые могут быть частью базы данных или отдельным файлом)
- ▶ *Количество внешних интерфейсных файлов.* Подсчитываются все логические файлы из других приложений, на которые ссылается данное приложение



Кейс 1: Пример подсчета функции данных

The image shows a web form for entering employee data. It is divided into two columns. The left column contains five input fields labeled: 'Фамилия' (Surname), 'Имя' (Name), 'Отчество' (Patronymic), 'Должность' (Position), and 'E-mail'. The right column contains three input fields labeled: 'ID сотрудника' (Employee ID), 'Код структурного подразделения' (Structural department code), and 'Зарплата' (Salary). A 'Сохранить' (Save) button is located at the bottom right of the form area.

Для описания типа «Сотрудник» используется 8 уникальных полей, следовательно **DET** = 8

Так как вся информация хранится в одном файле или таблице в базе данных, то **RET** = 1

Постановка задачи: Рассмотрим приложение, в котором имеется компонент системы, который отслеживает и хранит информацию о сотрудниках организации. Информация о каждом сотруднике описывается следующими полями: фамилия, имя, отчество, должность, адрес электронной почты, идентификационный номер сотрудника, код структурного подразделения и размер заработной платы. Все эти данные хранятся в пределах приложения в одном файле базы данных с именем «Сотрудник». Учитывая эту информацию, данная транзакция данных будет идентифицироваться как внутренний логический файл. Определить для него количество DET и RET

Если предположить, что информация о зарплате, соответствующей данной должности согласно штатному расписанию для указанного структурного подразделения, должна храниться во вторичной таблице с кодом структурного подразделения в качестве внешнего ключа, то количество **RET** будет равно 2



Кейс 2: Пример подсчета функции транзакции

Фамилия	ID сотрудника
Имя	Код структурного подразделения
Отчество	Зарплата
Должность	
E-mail	
Сохранить	

Форма содержит 8 уникальных полей, для оценки которых используется 8 DET.

Есть также еще один элемент, который считается как DET на данном экране. Это кнопка «Сохранить», нажатие на которую фактически инициирует транзакцию, позволяя системе поддерживать внутренний файл (отправляет данные в базу данных для безопасного хранения).

Итак, **общее количество DET** на этом экране равно **9**

Из приведенной экранной формы следует, что она позволяет пользователю приложения создать нового сотрудника в приложении. Наличие кнопки «Сохранить» позволяет идентифицировать этот экран как входной, для которого нам нужно подсчитать количество DET и FTR

Все введенные с помощью указанной формы данные относятся к информации об одном сотруднике в базе данных, следовательно при нажатии кнопки «Сохранить» они будут «перенесены» в одну запись во внутреннем файле, поэтому **FTR = 1**



Кейс 3: Взаимодействие с внешними файлами

The screenshot shows a light gray rectangular form with a thin black border. On the left side, there are five text input fields stacked vertically, each with a label above it: 'Фамилия' (Family name), 'Имя' (Name), 'Отчество' (Patronymic), 'Должность' (Position), and 'E-mail'. On the right side, there are two more text input fields stacked vertically, labeled 'ID сотрудника' (Employee ID) and 'Код структурного подразделения' (Department code). Below these, there is a larger text input field labeled 'Зарплата' (Salary). At the bottom right of the form is a gray button with the text 'Сохранить' (Save) in white.

Примечание: поля «Зарплата» и «Код структурного подразделения» доступны только для чтения

Учитывая данную информацию, транзакция данных теперь включает в себя 2 файла – внутренний файл, который поддерживает наше приложение, и внешний файл, в котором 2 поля «выбираются» из системы бухучета, которая отвечает за хранение указанных данных. Таким образом мы определяем, что **внутренний файл** имеет **1 RET** и **6 DET**, а **внешний файл** – **1 RET** и **2 DET**

И для **внутреннего логического файла**, и для **внешнего интерфейсного файла**, количество **FTR** будет равно **1**

Предположим, что данные для полей «Зарплата» и «Код структурного подразделения» теперь будут поставляться в нашу систему с помощью существующих данных, которые хранятся в системе бухгалтерского учета компании, вместо того, чтобы требовать от пользователя ввести эти данные. Номер сотрудника (ID), который пользователь вводит в нашу систему, заставит транзакцию «получить» необходимые данные из бухгалтерской системы. Мы по-прежнему предполагаем, что все данные будут храниться в пределах оцениваемого приложения, в одном файле базы данных с именем «Сотрудник»

Кейс 4: Подсчет функции транзакции для запроса

Фамилия	ID сотрудника
Имя	Код структурного подразделения
Отчество	Зарплата
Должность	
E-mail	
Сохранить	

Запрос также состоит из **1 FTR**, так как мы предполагаем, что **2 DET**, «выбранные» из системы бухгалтерского учета, содержатся в одной таблице этой системы

Если значения полей «Зарплата» и «Код структурного подразделения» выбираются из системы бухгалтерского учета компании на основании ID сотрудника, то когда пользователь нажимает кнопку «Сохранить» происходит обращение к этой внешней системе. Такой тип транзакции называется внешним запросом. В этом случае нажатие кнопки сохранения запускает две транзакции: во-первых, система обращается к системе бухучета и извлекает из нее некоторые данные, во-вторых, сохраняет данные, введенные конечным пользователем, и данные, полученные извне в одном файле (FTR = 1). Подобно счетчику функций данных (кейс 3), счетчик функций транзакций теперь разделен на две части: внешний ввод и внешний запрос



Порядок расчета трудоемкости разработки ПО

- ▶ определение количества функциональных типов данных для приложения
- ▶ определение количества связанных с каждым функциональным типом элементарных данных (DET), элементарных записей (RET) и файлов типа ссылок (FTR)
- ▶ определение сложности (в зависимости от количества DET, RET и FTR)
- ▶ подсчет количества функциональных точек приложения
- ▶ подсчет количества функциональных точек с учетом общих характеристик системы
- ▶ оценка трудоемкости разработки (с использованием различных статистических данных)



Определение количества и сложности функциональных типов для транзакций

- ▶ Количество функциональных типов для транзакций (входных элементов приложения, выходных элементов приложения и внешних запросов) определяется на основе выявления входных и выходных документов, экранных форм, отчетов, а также по диаграммам классов
- ▶ Далее для каждого выявленного функционального типа (EI, EO или EQ) определяется его сложность (низкая, средняя или высокая), которая зависит от количества связанных с этим функциональным типом DET, RET и FTR

Еще раз про DET, RET, FTR

- ▶ **DET** – это уникальное распознаваемое пользователем, нерекурсивное (неповторяющееся) поле данных
- ▶ **RET** – самая большая идентифицируемая пользователем логическая группа данных внутри ILF или EIF
- ▶ **FTR** – это тип файла, на который ссылается транзакция. FTR позволяет подсчитать количество различных файлов (информационных объектов) типа ILF и/или EIF модифицируемых или считываемых в транзакции



Ранг и оценка сложности внутренних логических файлов

Типы элементов-записей (RET)	Элементы данных (DET)		
	1-19	20-50	>50
1	Низкий (7)	Низкий (7)	Средний (10)
2-5	Низкий (7)	Средний (10)	Высокий (15)
>5	Средний (10)	Высокий (15)	Высокий (15)

Ранг и оценка сложности внешних интерфейсных файлов

Типы элементов-записей (RET)	Элементы данных (DET)		
	1-19	20-50	>50
1	Низкий (5)	Низкий (5)	Средний (7)
2-5	Низкий (5)	Средний (7)	Высокий (10)
>5	Средний (7)	Высокий (10)	Высокий (10)



Правила расчета DET для внешних вводов

В качестве DET (data element types) для внешних вводов (EI) учитываются:

- ▶ каждое нерекурсивное поле, принадлежащее внутреннему логическому файлу (ILF) (или поддерживаемое им) и обрабатываемое во вводе
- ▶ каждое поле, которое пользователь хотя и не вызывает, но оно через процесс ввода поддерживается во внутреннем логическом файле (ILF)
- ▶ логическое поле, которое физически представляет собой множество полей, но воспринимается пользователем как единый блок информации
- ▶ группа полей, которые появляются во внутреннем логическом файле (ILF) более одного раза, но в связи с особенностями алгоритма их использования воспринимаются как один DET
- ▶ группа полей, которые фиксируют ошибки в процессе обработки или подтверждают, что обработка закончилась успешно
- ▶ действие, которое может быть выполнено во вводе

Ранг и оценка сложности внешних вводов

Ссылки на файлы (FTR)	Элементы данных (DET)		
	1-4	5-15	>15
0-1	Низкий (3)	Низкий (3)	Средний (4)
2	Низкий (3)	Средний (4)	Высокий (6)
>2	Средний (4)	Высокий (6)	Высокий (6)



Правила расчета DET для внешних выводов

В качестве DET для внешних выводов (ЕО) учитываются:

- ▶ каждое распознаваемое пользователем нерекурсивное поле, участвующее в процессе вывода
- ▶ поле, которое физически отображается в виде нескольких полей его составляющих, но используется как единый информационный элемент
- ▶ каждый тип метки и каждое значение числового эквивалента при графическом выводе
- ▶ текстовая информация, которая может содержать одно слово, предложение или фразу

Примечание: переменные, определяющие номера страниц или генерируемые системой логотипы, не являются элементами данных

Ранг и оценка сложности внешних выводов

Ссылки на файлы (FTR)	Элементы данных (DET)		
	1-4	5-19	>19
0-1	Низкий (4)	Низкий (4)	Средний (5)
2-3	Низкий (4)	Средний (5)	Высокий (7)
>3	Средний (5)	Высокий (7)	Высокий (7)



Правила расчета DET для внешних запросов

В качестве DET для внешнего запроса (EQ) **по входу** учитываются:

- ▶ каждое распознаваемое пользователем нерекурсивное поле, появляющееся во вводной части запроса
- ▶ каждое поле, которое определяет критерий выбора данных
- ▶ группа полей, в которых выдаются сообщения о возникающих ошибках в процессе ввода информации или подтверждающих успешное завершение процесса ввода
- ▶ группа полей, которые позволяют выполнять запросы

В качестве DET для внешнего запроса (EQ) **по выходу** учитываются:

- ▶ каждое распознаваемое пользователем нерекурсивное поле, которое появляется в выводной части запроса
- ▶ логическое поле, которое физически отображается как группа полей, однако воспринимается пользователем как единое поле
- ▶ группа полей, которые в соответствии с методикой обработки могут повторяться во внутреннем логическом файле (ILF)

Примечание: колонтитулы или генерируемые системой иконки не учитываются как DET



Ранг и оценка сложности внешних запросов

Ссылки на файлы (FTR)	Элементы данных		
	1-4	5-19	>19
0-1	Низкий (3)	Низкий (3)	Средний (4)
2-3	Низкий (3)	Средний (4)	Высокий (6)
>3	Средний (4)	Высокий (6)	Высокий (6)

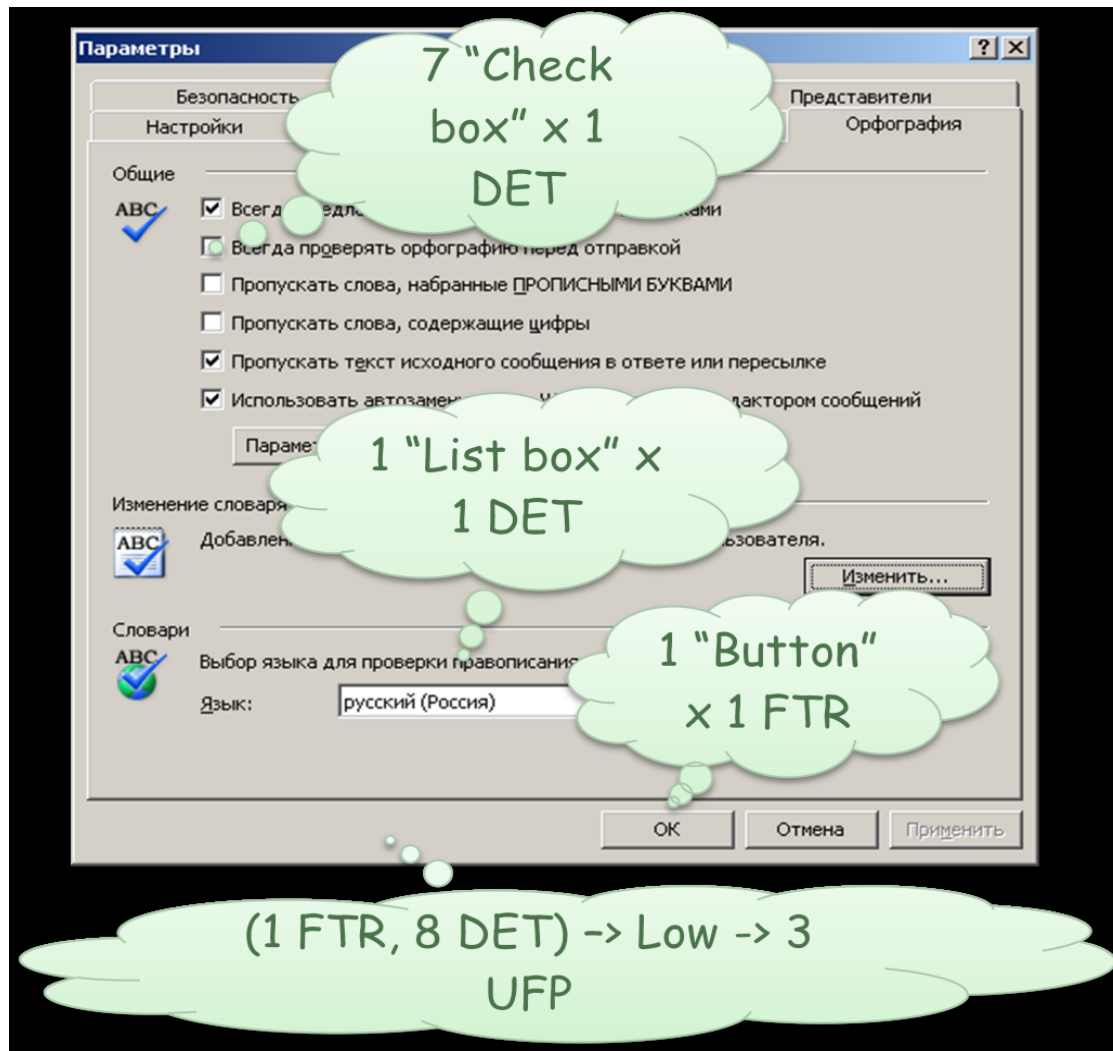


Учет элементов данных из графического интерфейса пользователя

Элемент данных	Правило учета
Группа радиокнопок	Так как в группе пользователь выбирает только одну радиокнопку, все радиокнопки группы считаются одним элементом данных
Группа флажков (переключателей)	Так как в группе пользователь может выбрать несколько флажков, каждый флажок считают элементом данных
Командные кнопки	Командная кнопка может определять действие добавления, изменения или запроса. Кнопка ОК может вызывать транзакции различных типов. Каждая кнопка считается отдельным элементом данных
Списки	Список может быть внешним запросом, но результат запроса может быть элементом данных внешнего ввода



Диалоговое окно, управляющее проверкой орфографии в MS Office Outlook



Источник: С. Архипенков. Лекции по управлению программными проектами, стр. 102

Алгоритм применения метода функциональных точек

- ▶ Подсчитываются функции для каждой категории (вводы, выводы, запросы, структуры данных, интерфейсы)
- ▶ Устанавливаются требования для каждой категории
- ▶ Определяется сложность каждой функции (высокая, средняя, низкая)
- ▶ Каждая функция умножается на соответствующий ей параметр, а затем все значения суммируются с целью получения общего количества функциональных точек



Исходные данные для расчета общего количества функциональных точек

Имя характеристики	Ранг, сложность, количество			
	Низкий	Средний	Высокий	Итого
Внешние вводы	__x3 = __	__x4 = __	__x6 = __	= __
Внешние выводы	__x4 = __	__x5 = __	__x7 = __	= __
Внешние запросы	__x3 = __	__x4 = __	__x6 = __	= __
Внутренние логические файлы	__x7 = __	__x 1__ = __	__x15 = __	= __
Внешние интерфейсные файлы	__x5 = __	__x7 = __	__x1__ = __	= __
Общее количество				= __



Кейс 5: Примеры анализа данных

Проектируется GUI для обслуживания клиентов, который будет иметь поля: *Имя, Адрес, Город, Страна, Почтовый Индекс, Телефон, Email* (таким образом, имеется 7 полей или 7 DET)

Кроме того, предусматриваются 3 командные кнопки: *Добавить, Изменить, Удалить*. Следовательно каждый из внешних вводов *Добавить, Изменить, Удалить* будет состоять из 8 DET (7 полей плюс командная кнопка)

В данном приложении генерируются 3 типа сообщений: *сообщения об ошибке, сообщения подтверждения и сообщения уведомления*. Сообщения об ошибке (например, *Неверный пароль*) и сообщения подтверждения (например, *Вы действительно хотите удалить клиента?*) указывают, что произошла ошибка или что процесс может быть завершен. Эти сообщения не образуют самостоятельного процесса, они являются частью другого процесса, то есть считаются элементом данных соответствующей транзакции

Уведомление является независимым элементарным процессом. Например, при попытке получить из банкомата сумму денег, превышающую их количество на счете, генерируется сообщение *Не хватает средств для завершения транзакции*. Оно является результатом чтения информации из файла счета и формирования заключения. Сообщение уведомления рассматривается как внешний вывод



Скорректированное количество функциональных точек

$FP = \text{Общее количество} * (0,65 + 0,01 * \sum Fi),$

где Fi — 14 коэффициентов регулировки сложности

Каждый коэффициент может принимать следующие значения:

- 0 — нет влияния,
- 1 — случайное,
- 2 — небольшое,
- 3 — среднее,
- 4 — важное,
- 5 — основное



Определение системных параметров приложения

№	Системный параметр	Описание
1	Передача данных	Сколько средств связи требуется для передачи или обмена информацией с приложением или системой?
2	Распределенная обработка данных	Как обрабатываются распределенные данные и функции обработки?
3	Производительность	Нуждается ли пользователь в фиксации времени ответа или производительности?
4	Эксплуатационные ограничения	Насколько сильны эксплуатационные ограничения и каков объем специальных усилий на их преодоление?
5	Частота транзакций	Как часто выполняются транзакции (каждый день, каждую неделю, каждый месяц) ?
6	Оперативный ввод данных	Какой процент информации надо вводить в режиме онлайн?
7	Эффективность работы конечных пользователей	Приложение проектировалось для обеспечения эффективной работы конечного пользователя?
8	Оперативное обновление	Как много внутренних файлов обновляется в онлайн-транзакции?
9	Сложность обработки	Выполняет ли приложение интенсивную логическую или математическую обработку?
10	Повторная используемость	Приложение разрабатывалось для удовлетворения требований одного или многих пользователей?
11	Легкость инсталляции	Насколько трудны преобразование и инсталляция приложения?
12	Легкость эксплуатации	Насколько эффективны и/или автоматизированы процедуры запуска, резервирования и восстановления?
13	Количество возможных установок на различных платформах	Была ли спроектирована, разработана и поддержана возможность инсталляции приложения в разных местах для различных организаций?
14	Простота изменений (гибкость)	Была ли спроектирована, разработана и поддержана в приложении простота изменений?

Передачи данных

Значение	Описание
0	Полностью пакетная обработка на локальном ПК
1	Пакетная обработка, удаленный ввод данных или удаленная печать
2	Пакетная обработка, удаленный ввод данных и удаленная печать
3	Сбор данных в режиме «онлайн» или дистанционная обработка, связанная с пакетным процессом
4	Несколько внешних интерфейсов, один тип коммуникационного протокола
5	Несколько внешних интерфейсов, несколько типов коммуникационных протоколов



Распределенная обработка данных

Значение	Описание
0	Передача данных или процессов между компонентами системы отсутствует
1	Приложение готовит данные для обработки на ПК конечного пользователя
2	Данные готовятся для передачи, затем передаются и обрабатываются на другом компоненте системы (не на ПК конечного пользователя)
3	Распределенная обработка и передача данных в режиме «онлайн» только в одном направлении
4	Распределенная обработка и передача данных в режиме «онлайн» в обоих направлениях
5	Динамическое выполнение процессов в любом подходящем компоненте системы



Производительность

Значение	Описание
0	К системе не предъявляется специальных требований, касающихся производительности
1	Требования к производительности определены, но не требуется никаких специальных действий
2	Время реакции или пропускная способность являются критическими в пиковые периоды. Не требуется никаких специальных решений относительно использования ресурсов процессора. Обработка может быть завершена в течение следующего рабочего дня
3	Время реакции или пропускная способность являются критическими в обычное рабочее время. Не требуется никаких специальных решений относительно использования ресурсов процессора. Время обработки ограничено взаимодействующими системами
4	То же, кроме того, пользовательские требования к производительности достаточно серьезны, чтобы ее необходимо было анализировать на стадии проектирования
5	То же, кроме того, на стадиях проектирования, разработки и (или) реализации для удовлетворения пользовательских требований к производительности используются специальные средства анализа



Эксплуатационные ограничения

Значение	Описание
0	Какие-либо явные или неявные ограничения отсутствуют
1	Эксплуатационные ограничения присутствуют, но не требуют никаких специальных усилий
2	Должны учитываться некоторые ограничения, связанные с безопасностью или временем реакции
3	Должны учитываться конкретные требования к процессору со стороны конкретных компонентов приложения
4	Заданные эксплуатационные ограничения требуют специальных ограничений на выполнение приложения в центральном или выделенном процессоре
5	То же, кроме того, специальные ограничения затрагивают распределенные компоненты системы



Частота транзакций

Значение	Описание
0	Пиковых периодов не ожидается
1	Ожидаются пиковые периоды (ежемесячные, ежеквартальные, ежегодные)
2	Ожидаются еженедельные пиковые периоды
3	Ожидаются ежедневные пиковые периоды
4	Высокая частота транзакций требует анализа производительности на стадии проектирования
5	То же, кроме того, на стадиях проектирования, разработки и (или) внедрения необходимо использовать специальные средства анализа производительности



Оперативный ввод данных

Значение	Описание
0	Все транзакции обрабатываются в пакетном режиме
1	От 1% до 7% транзакций требуют интерактивного ввода данных
2	От 8% до 15% транзакций требуют интерактивного ввода данных
3	От 16% до 23% транзакций требуют интерактивного ввода данных
4	От 24% до 30% транзакций требуют интерактивного ввода данных
5	Более 30% транзакций требуют интерактивного ввода данных



Эффективность работы конечных пользователей

Эффективность работы конечных пользователей определяется наличием следующих функциональных возможностей:

- ▶ Средства навигации (например, функциональные клавиши, динамически генерируемые меню)
- ▶ Меню
- ▶ Онлайн-подсказки и документация
- ▶ Автоматическое перемещение курсора
- ▶ Скроллинг
- ▶ Удаленная печать
- ▶ Предварительно назначенные функциональные клавиши
- ▶ Выбор данных на экране с помощью курсора
- ▶ Использование видеоэффектов, цветового выделения, подчеркивания и других индикаторов
- ▶ Всплывающие окна
- ▶ Минимизация количества экранов, необходимых для выполнения бизнес-функций
- ▶ Поддержка двух и более языков



Эффективность работы конечных пользователей

Значение	Описание
0	Ни одной из перечисленных функциональных возможностей
1	От одной до трех функциональных возможностей
2	От четырех до пяти функциональных возможностей
3	Шесть или более функциональных возможностей при отсутствии конкретных пользовательских требований к эффективности
4	То же, кроме того, пользовательские требования к эффективности требуют специальных проектных решений для учета эргономических факторов (например, минимизации нажатий клавиш, максимизации значений по умолчанию, использования шаблонов)
5	То же, кроме того, пользовательские требования к эффективности требуют применения специальных средств и процессов, демонстрирующих их выполнение



Оперативное обновление

Значение	Описание
0	Отсутствует
1	Онлайновое обновление от одного до трех управляющих файлов Объем обновлений незначителен, восстановление несложно
2	Онлайновое обновление четырех или более управляющих файлов Объем обновлений незначителен, восстановление несложно
3	Онлайновое обновление основных внутренних логических файлов
4	То же, плюс необходимость специальной защиты от потери данных
5	То же, кроме того, большой объем данных требует учета затрат на процесс восстановления. Требуются автоматизированные процедуры восстановления с минимальным вмешательством оператора



Сложность обработки

Сложность обработки характеризуется наличием у приложения следующих функциональных возможностей:

- ▶ повышенная реакция на внешние воздействия и (или) специальная защита от внешних воздействий
- ▶ экстенсивная логическая обработка
- ▶ экстенсивная математическая обработка
- ▶ обработка большого количества исключительных ситуаций
- ▶ поддержка разнородных типов входных/выходных данных

Значение	Описание
0	Ни одной из перечисленных функциональных возможностей
1	Любая одна из возможностей
2	Любые две возможности
3	Любые три возможности
4	Любые три возможности
5	Все пять возможностей



Повторное использование

Значение	Описание
0	Отсутствует
1	Повторное использование кода внутри одного приложения
2	Не более 10% приложений будут использоваться более чем одним пользователем
3	Более 10% приложений будут использоваться более чем одним пользователем
4	Приложение оформляется как продукт и (или) документируется для облегчения повторного использования. Настройка приложения выполняется пользователем на уровне исходного кода
5	То же, с возможностью параметрической настройки приложений



Легкость инсталляции

Значение	Описание
0	К установке не предъявляется никаких специальных требований
1	Для установки требуется специальная процедура
2	Заданы пользовательские требования к конвертированию (переносу существующих данных и приложений в новую систему) и установке, должны быть обеспечены и проверены соответствующие руководства. Конвертированию не придается важное значение
3	То же, однако конвертированию придается важное значение
4	То же, что и в случае 2, плюс наличие автоматизированных средств конвертирования и установки
5	То же, что и в случае 3, плюс наличие автоматизированных средств конвертирования и установки



Легкость эксплуатации

Значение	Описание
0	К эксплуатации не предъявляется никаких специальных требований, за исключением обычных процедур резервного копирования
1-4	Приложение обладает одной, несколькими или всеми из перечисленных далее возможностей. Каждая возможность, за исключением второй, обладает единичным весом: 1) наличие процедур запуска, копирования и восстановления с участием оператора; 2) то же, без участия оператора; 3) минимизируется необходимость в монтировании носителей для резервного копирования; 4) минимизируется необходимость в средствах подачи и укладки бумаги при печати
5	Вмешательство оператора требуется только при запуске и завершении работы системы. Обеспечивается автоматическое восстановление работоспособности приложения после сбоев и ошибок



Количество возможных установок на различных платформах

Значение	Описание
0	Приложение рассчитано на установку у одного пользователя
1	Приложение рассчитано на много установок для строго стандартной платформы (технические средства + программное обеспечение)
2	Приложение рассчитано на много установок для платформ с близкими характеристиками
3	Приложение рассчитано на много установок для различных платформ
4	То же, что в случаях 1 или 2, плюс наличие документации и планов поддержки всех установленных копий приложения
5	То же, что в случае 3, плюс наличие документации и планов поддержки всех установленных копий приложения



Простота изменений (гибкость)

Гибкость характеризуется наличием у приложения следующих возможностей:

- ▶ поддержка простых запросов, например, логики и (или) в применении только к одному внутреннему логическому файлу (ILF) (вес — 1)
- ▶ поддержка запросов средней сложности, например, логики и (или) в применении более чем к одному ILF (вес - 2)
- ▶ поддержка сложных запросов, например, комбинации логических связей и (или) в применении к одному или более ILF (вес — 3)
- ▶ управляющая информация хранится в таблицах, поддерживаемых пользователем в интерактивном режиме, однако эффект от ее изменений проявляется на следующий рабочий день
- ▶ то же, но эффект проявляется немедленно (вес — 2)

Значение	Описание
0	Ни одной из перечисленных функциональных возможностей
1	Любая одна из возможностей
2	Любые две возможности
3	Любые три возможности
4	Любые три возможности
5	Все пять возможностей



Формула расчета трудоемкости

Трудоемкость (трудозатраты) = $FP * P$

где:

FP — количество функциональных точек

P — средние затраты времени на реализацию одной точки (индекс производительности)

Индекс производительности индивидуален для каждой команды. Ниже приведены усредненные рыночные данные (в среднем по индустрии):

- ▶ Высокая производительность: 4–7 чел.-часов на 1 FP (опытная команда, готовые фреймворки)
- ▶ Средняя производительность: 8–12 чел.-часов на 1 FP (стандарт для большинства проектов)
- ▶ Низкая производительность: 15–20+ чел.-часов на 1 FP (сложные системы, жесткое регулирование, новички в разработке)

Пример: Если при оценке системы мы получили значение 25 FP, то трудозатраты опытной команды составят $25 * 6 = 150$ часов, команде средней квалификации для реализации той же функциональности понадобится $25 * 10 = 250$ часов, у новичков на ту же работу уйдет $25 * 15 = 375$ часов



Базовая (эмпирическая) формула расчета времени

Для оценки времени выполнения проекта среднего размера применяется формула, основанная на трудоемкости в человеко-месяцах:

$$\text{Срок (в месяцах)} = 3.0 * \sqrt[3]{\text{Трудоемкость (в человеко-месяцах)}}$$

Формула расчета времени, исходя из количества исполнителей

В этом случае используется формула, учитывающая помимо количества людей еще так называемый коэффициент коммуникаций:

$$\text{Срок} = \frac{\text{Трудоемкость (в часах)}}{\text{Кол—во человек} * \text{Часы в день} * 0.7}$$

где 0.7 (или 70%) — это коэффициент полезного действия. Остальные 30% времени уходят на совещания, переписку, перекуры и уточнение требований

Примечание: Обычно расчет по функциональным точкам покрывает только разработку. Необходимо заложить еще +20-30% времени на тестирование и устранение ошибок и подготовку релизной версии



Пересчет FP-оценок в LOC-оценки

Язык программирования	Количество операторов на один FP
Ассемблер	320
C	128
Кобол	106
Фортран	105
Паскаль	91
C++	53
C#	53
Java	53
JavaScript	47

Язык программирования	Количество операторов на один FP
Python	47
Ada 95	49
Visual Basic	32
Visual C++	34
Delphi 5	18
HTML	40
Prolog	64
SQL	21
Lisp	64



Размер программного обеспечения в FP и LOC

Тип ПО	Размер, FP	Размер, LOC	Количество LOC на одну FP
Текстовые процессоры	3500	437500	125
Электронные таблицы	3500	437500	125
Клиент-серверные приложения	7500	675000	90
ПО баз данных	7500	937500	125
Производственные приложения	7500	937500	125
Крупные бизнес-приложения	10000	1050000	105
Операционные системы	75000	11250000	150
Системы масштаба предприятий	150000	18750000	125
Крупные оборонные системы	250000	25000000	100



Распределение временных затрат по стадиям для маленьких и больших проектов

Масштаб проекта	Начальная стадия, %	Проектирование, %	Разработка, %	Ввод в действие, %
Маленький коммерческий проект	10	20	50	20
Большой, сложный проект	15	30	40	15



Статистические данные по времени разработки

Размер проекта	< 100 FP	100 - 1000 FP	1000 - 10000 FP	> 10000 FP
Планируемый срок (мес.)	6	12	18	24
Реальный срок (мес.)	8	16	24	36
Отставание	2	4	6	12

Примечание: причины отставания частично объясняются неточной оценкой, частично — ростом количества требований к системе после того, как выполнена начальная оценка



Производительность разработчиков для стандартных типов проектов

Тип программы	Строк кода на человеко-месяц (номинал)		
	проект на 1000 LOC	проект на 10000 LOC	проект на 25000 LOC
Авиационное оборудование	100 - 1000 (200)	20 - 300 (50)	20 - 200 (40)
Бизнес-система	800 - 18000 (3000)	200 - 7000 (600)	100 - 5000 (500)
Системы управления	200 - 3000 (500)	50 - 600 (100)	40 - 500 (80)
Встроенные системы	100 - 2000 (300)	30 - 500 (70)	20 - 400 (60)
Открытые Интернет-системы	600 - 10000 (1500)	100 - 2000 (300)	100 - 1500 (200)
Микрокод	100 - 800 (200)	20 - 200 (40)	20 - 100 (30)
Управление процессами	500 - 5000 (1000)	100 - 1000 (300)	80 - 900 (200)
Системы реального времени	100 - 1500 (200)	20 - 300 (50)	20 - 300 (40)
Системы научных и инженерных исследований	500 - 7500 (1000)	100 - 1500 (300)	80 - 1000 (200)
Коммерческие пакеты	400 - 5000 (1000)	100 - 1000 (200)	70 - 800 (200)
Системные программы/драйверы	200 - 5000 (600)	50 - 1000 (100)	40 - 800 (90)
Телекоммуникации	200 - 3000 (600)	50 - 600 (100)	40 - 500 (90)



ОСНОВНЫЕ ВЫВОДЫ

- ▶ Оценка трудоемкости должна быть вероятностным утверждением. Это означает, что для нее существует некоторое распределение вероятности, которое может быть очень широким, если неопределенность высокая, или достаточно узким, если неопределенность низкая
- ▶ Использование собственного опыта или опыта коллег, полученного в похожих проектах, это наиболее прагматичный подход, который позволяет получить достаточно реалистичные оценки трудоемкости и срока реализации программного проекта, быстро и без больших затрат
- ▶ Если собственный опыт аналогичных проектов отсутствует, а экспертное мнение недоступно, то необходимо использовать формальные методики, основанные на обобщенном отраслевом опыте, а именно метод функциональных точек или модель COSOMO II
- ▶ Недооценка приводит к ошибкам планирования и неэффективному взаимодействию. Агрессивные сроки, постоянное давление, сверхурочные, авралы служат причиной того, что затраты на проект растут экспоненциально и неограниченно





Спасибо за внимание!

Пример расчета количества функциональных точек

Источник: <https://topuch.com/laboratornaya-rabota-1-2-ocenka-razmera-i-slojnosti-programmni/index3.html#pages>

Рассмотрим пример расчета количества функциональных точек для ПС, реализующего функции телефонного справочника. Допустим, что в справочнике необходимо хранить номер телефона, фамилию и инициалы владельца, а также его адрес. Должны быть предусмотрены возможности поиска записей и сортировки списка. Внешний вид единственной экранной формы ПС приведен на рис. 1.

Произведем подсчет количества функциональных точек.

1. Установление границ данного ПС не вызывает трудностей, так как оно является полностью локальным, и обмен данными с другими ПС в нем не предусматривается.
2. В приложении имеется один внутренний логический файл (ILF) для хранения информации справочника. Причем, данные могут храниться как в обычном файле, так и в таблице СУБД.

Телефон	Фамилия И.О.	Адрес

Добавить
Удалить

Сортировка
☐ Телефон
☐ Фамилия
☒ Адрес

Поиск
Телефон Фамилия Адрес
Поиск

Выход

Рис. 1. Экранная форма телефонного справочника.

3. Число типов элементов записей (RET) для этого файла может быть равно единице, если данные в файле хранятся в виде однотипных записей: «Телефон», «Фамилия» и «Адрес», допустим, представлены в символьном формате. В случае же, если номер телефона будет представлен, как целое число, а фамилия и адрес в символьном формате, то тогда внутренний логический файл будет иметь два RET. Для определенности далее будем считать, что внутренний логический файл имеет два RET.
4. Число типов элементов данных (DET) внутреннего логического файла будет равно трем вне зависимости от формата представления номера

- телефона («Телефон», «Фамилия», «Адрес»). Таким образом, уровень сложности внутреннего логического файла – низкий.
5. Внешних интерфейсных файлов (EIF) данное ПС не имеет.
 6. В ПС имеются два внешних ввода (EI): «Добавление записи» и «Удаление записи», поскольку именно эти две функции ПС модифицируют данные во внутреннем логическом файле. Так как внешний ввод «Добавление записи» ссылается на один внутренний логический файл и имеет пять элементов данных (поля «Телефон», «Фамилия», «Адрес», кнопка «Добавить» и сообщение, подтверждающее факт добавления записи), то уровень сложности этого ввода низкий. Аналогично, уровень сложности внешнего ввода «Удаление записи» также низкий, поскольку имеется один FTR и пять DET (поля «Телефон», «Фамилия», «Адрес», кнопка «Удалить» и сообщение, подтверждающее факт удаления записи).
 7. В программе имеются два внешних запроса (EQ): «Вывод списка» отсортированных записей и «Поиск записи» в справочнике. Внешний запрос «Вывод списка» имеет низкий уровень сложности, так как ссылается на один внутренний логический файл и имеет четыре элемента данных («Телефон», «Фамилия», «Адрес» и группа радио-кнопок «Сортировка»). Уровень сложности «Поиска записи» в справочнике также низкий (один внутренний логический файл и пять элементов данных: «Телефон», «Фамилия», «Адрес», кнопка «Поиск», сообщение об отсутствии искомой информации).
 8. В ПС имеется также один внешний вывод (EO): вывод уведомляющего сообщения при попытке добавить запись с существующим номером телефона. Уровень сложности этого внешнего вывода – низкий, так как он имеет один FTR и два DET: номер телефона и само сообщение.

Полученные данные сведем в табл. 1 и рассчитаем ненормированное количество функциональных точек (UFPC).

Таблица 1. Данные для расчета числа UFPC телефонного справочника

Характеристика	Уровень сложности			Итого
	Кол-во	Низкий Ранг	Итог	
Внешние вводы (EI)	2	5	6	6
Внешние выводы (EO)	1	4	4	4
Внешние запросы (EQ)	2	3	6	6
Внутренние логические файлы (ILF)	1	7	7	7
Внешние интерфейсные файлы (EIF)	0	5	0	0
Итого (UFPC)				23

Оценка Saitistika по системе COCOMO

Введение

Модель COCOMO (COConstructive COst MOdel) разработана Барри Боэмом (директор USC Center for Software Engineering). Это одна из основных методик, которые применяются для оценки стоимости ПО. Среди других методик она выгодно отличается простотой расчетов.

Тезисы

- Общий объем программы - **170.000 строк**.
- Создание продукта, аналогичного Saitistika, по оптимальной традиционной модели разработки, потребует:

Параметр	Значение
Общий объем работы	872 человеко-месяца
! Создание руководств	52 человеко-месяца
Календарное время	35 месяца
! Только проектирование	18 месяцев
Среднее количество занятых работников	25
! Максимальное количество	47

Правила расчета

Тип системы: промежуточная

Формулы:

1. Работа = $3.0 * EAF * (KDSI)^{1.12}$
2. Время = $2.5 * (Работа)^{0.35}$

Расчет EAF

Фактор	Значение	Влияние
Знание языка программирования	Номинальное	1.0
Ограничение времени выполнения	Номинальное	1.0
Размер базы данных	Номинальное	1.0
Межремонтный срок службы компьютера	Номинальное	1.0
Знание виртуальной машины	Номинальное	1.0
Изменчивость виртуальной машины	Номинальное	1.0
Использование программных инструментов	Высокое	0.9
Использование современных методов	Номинальное	1.0

Ограничение объема памяти	Низкое	1.0
Знание приложений	Высокое	0.95
Ограничение по срокам разработки	Номинальное	1.0
Требуемая надежность	Номинальное	1.0
Сложность продукта	Номинальное	1.0
Способности персонала/команды	Номинальное	1.0
Способности аналитика	Номинальное	1.0

$$EAF=1.0^{13} \cdot 0.9 \cdot 0.95 = 0.855$$

Расчет работы и времени

$$\text{Работа} = 3.0 \cdot 0.855 \cdot (170)^{1.12} = 2.8 \cdot 0.855 \cdot 474 = 808 \text{ человеко-месяца}^*$$

$$\text{Время} = 2.5 \cdot (808)^{0.35} = 2.5 \cdot 9.5 = 26 \text{ месяцев}^*$$

* Работа и время **не включают** затраты на планирование и определение требований

Итоговые цифры:

- **Работа = 872 человеко-месяца**
- **Время = 35 месяцев**

Распределение работ и времени по стадиям жизненного цикла при традиционном подходе

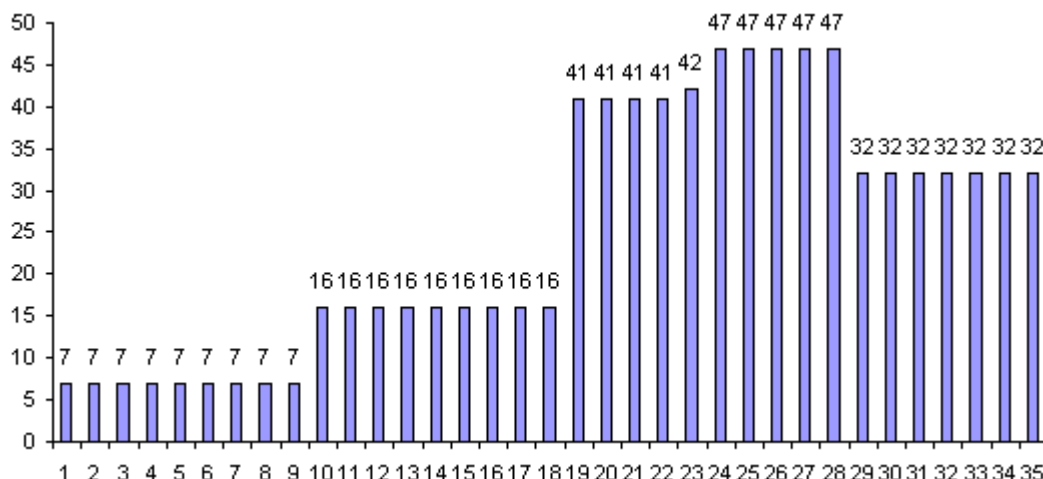
Вид деятельности	Работа (%)	Время (%)
Планирование и определение требований	(+8)	(+36)
Проектирование продукта	18	36
Детальное проектирование	25	18
Кодирование и тестирование отдельных модулей	26	18
Интеграция и тестирование	31	28
ИТОГО:	108%	136%

Стандартное распределение работ по видам деятельности WBS в модели СОСОМО

Тип кода	Бюджет (%)	Человеко-месяцы
Анализ требований	4	35
Проектирование продукта	12	105
Программирование	44	384
Планирование тестирования	6	52
Верификация и аттестация	14	122
Канцелярия проекта	7	61

Управление конфигурацией и обеспечение качества	7	61
Создание руководств	6	52
ИТОГО	100%	872

Количество работников на протяжении всего цикла создания продукта



Сравнения

Программа	Количество строк	Примечания
Saitistika	170 тыс.	Версия 4.0 beta
Communiware	80 тыс.	
Navision Axapta	400 тыс.	
Галактика	1.5 млн.	
SAP R/3	10 млн.	
SUN StarOffice 6	9 млн.	
Система управления огнем для истребителя F-16	236 тыс.	Стоила \$85 млн. Усилия по сопровождению программ комплекса, их улучшению и устранению ошибок потребовали дополнительно \$250 млн.
Компьютерная сеть Олимпийских игр в Солт Лейк Сити	10 млн.	Стоимость - \$300 млн.
Электронная система торгов NASDAQ	11 млн.	Только на поддержку в 2000 году потрачено \$55 млн.
Solaris 7	12 млн.	
Windows NT 4.0	16 млн.	
Компьютерная системе Агентства национальной безопасности США	25 млн.	
Программное обеспечение проекта НАСА "Space Shuttle"	40 млн.	

Windows 2000	40 млн.	
Debian Linux	55 млн.	Самый полный вариант, более 10 CD